第10章

行为型设计模式

陈龙彪
longbiaochen@xmu.edu.cn

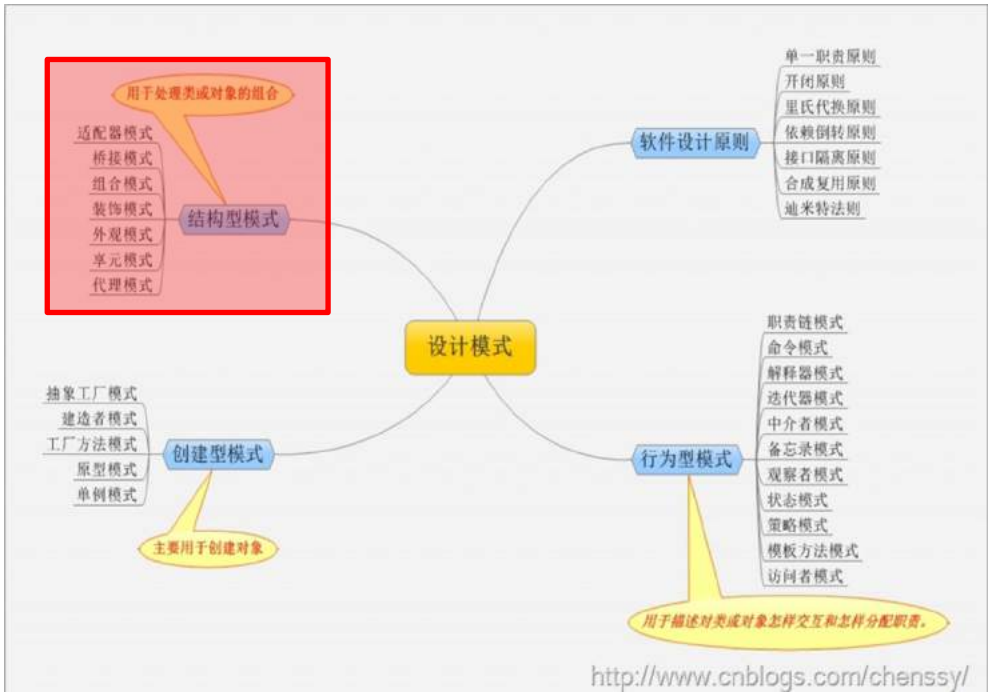
厦门大学信息学院
2019年4月

设计模式

范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	解释器模式 模板方法模式
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 享元模式 代理模式	职责链模式 命令模式 迭代器模式 中介者模式 备忘录模式 观察者模式 状态模式 策略模式 访问者模式

行为型模式

- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）



行为型模式



- 模式概述
  - 行为型模式（Behavioral Pattern）是对在不同的对象之间划分责任和算法的抽象化
    - 行为型模式不仅仅关注类和对象的结构，而且重点关注它们之间的相互作用
  - 通过行为型模式，可以清晰地划分类与对象的职责，并研究系统在运行时实例对象之间的交互
    - 在系统运行时，对象并不是孤立的，它们可以通过相互通信与协作完成某些复杂功能，一个对象在运行时也将影响到其他对象的运行

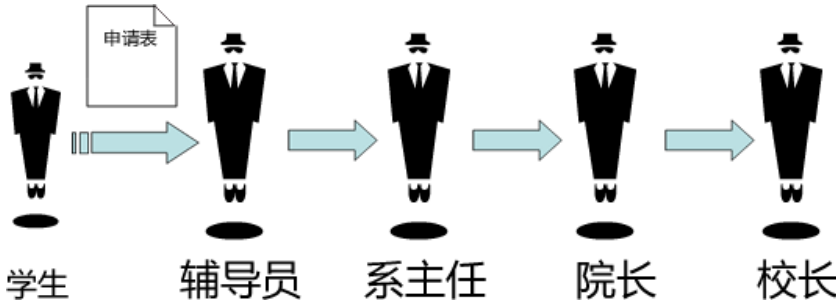
行为型模式

- 行为型模式概述
  - 类行为型模式：使用类的继承关系在几个类之间分配行为，类行为型模式主要通过多态等方式来分配父类与子类的职责
  - 对象行为型模式：使用对象的聚合关联关系来分配对象行为。根据“合成复用原则”，系统中要尽量使用关联关系来取代继承关系，因此大部分行为型设计模式都属于对象行为型设计模式

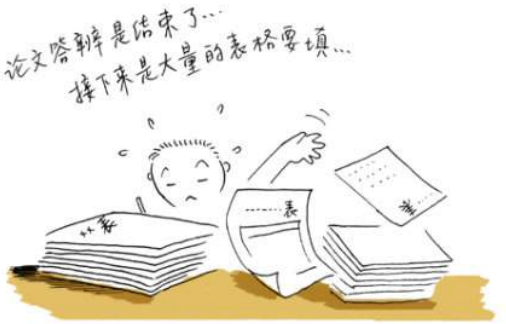


行为型模式

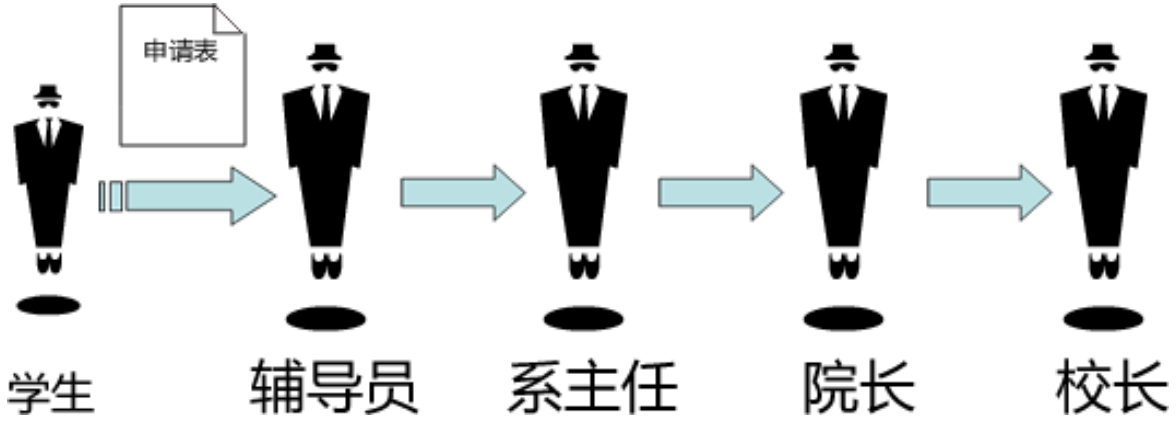
- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）



职责链模式



- 模式动机
  - 填申请表

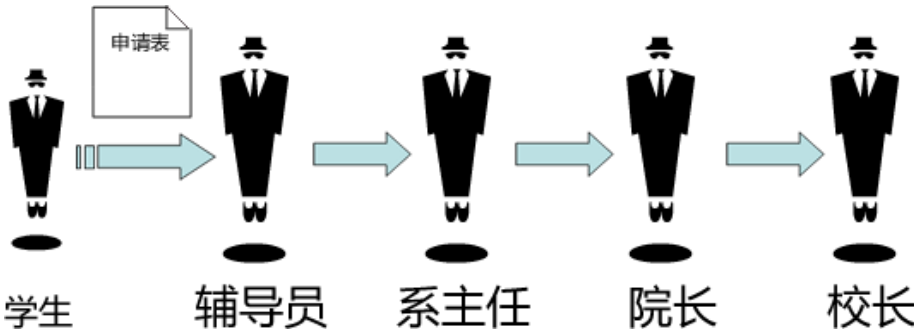




厦大软件体系结构课观光团：理发店的故事（一）

职责链模式

- 模式动机
  - 职责链可以是一条直线、一个环或者一个树形结构
    - 最常见的职责链是直线型，即沿着一条单向的链来传递请求
  - 链上的每一个对象都是请求处理者，职责链模式可以将请求的处理者组织成一条链，并使请求沿着链传递，由链上的处理者对请求进行相应的处理，客户端无须关心请求的处理细节以及请求的传递，只需将请求发送到链上即可，将请求的发送者和请求的处理者解耦。这就是职责链模式的模式动机。



职责链模式

- 模式定义
  - 职责链模式（Chain of Responsibility Pattern）避免请求发送者与接收者耦合在一起，让多个对象都有可能接收请求，将这些对象连接成一条链，并且沿着这条链传递请求，直到有对象处理它为止。职责链模式是一种对象行为型模式。

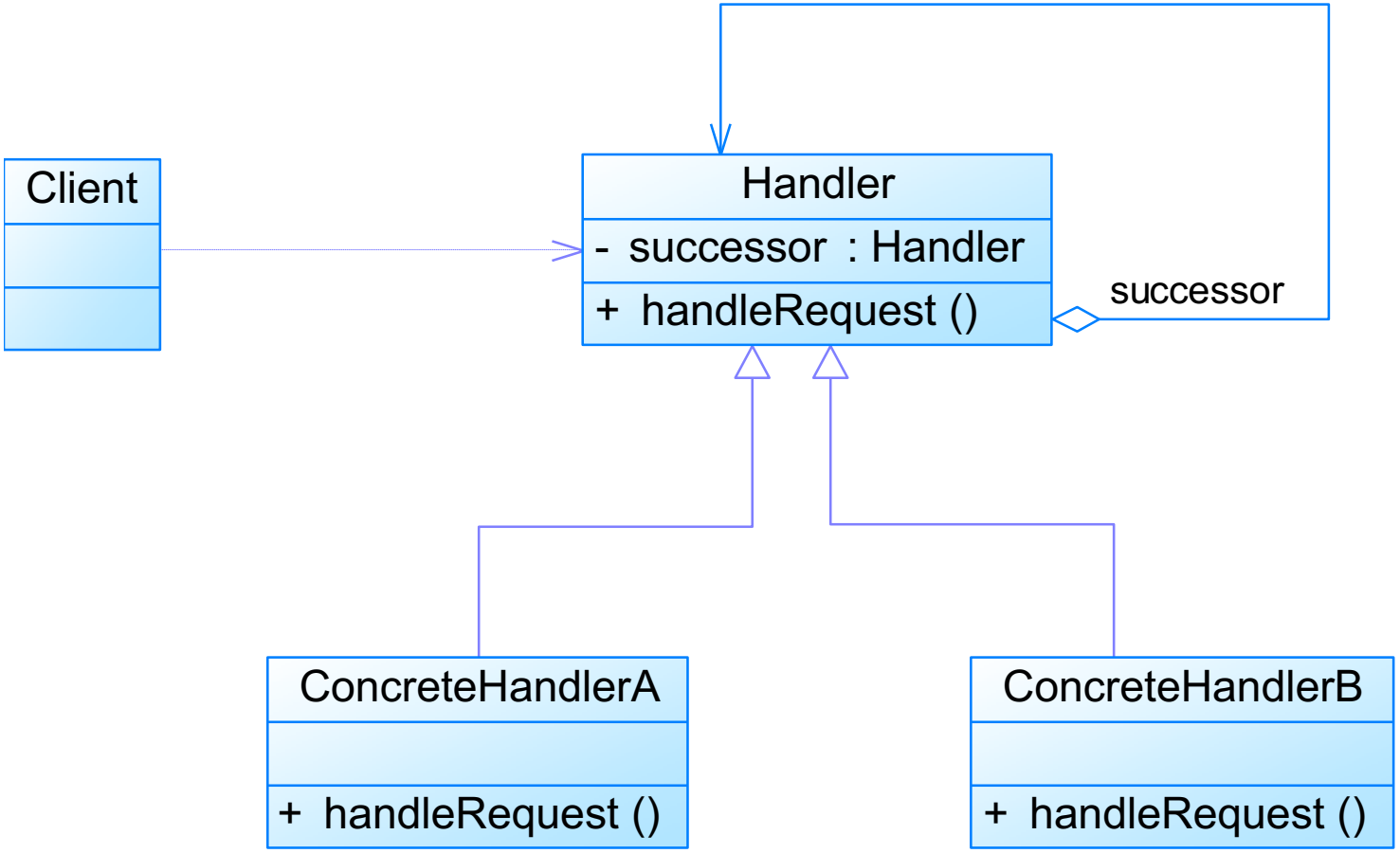


职责链模式

职责链模式包含如下角色：

- Handler: 抽象处理者
- ConcreteHandler: 具体处理者
- Client: 客户类

模式结构



职责链模式

- 模式分析
  - 典型的抽象处理者代码：

```
public abstract class Handler {
    protected Handler successor;

    public void setSuccessor(Handler successor) {
        this.successor = successor;
    }

    public abstract void handleRequest(String request);
}
```

职责链模式

- 模式分析
  - 典型的具体处理者代码：

```
public class ConcreteHandler extends Handler {
    public void handleRequest(String request) {
        if(请求 request 满足条件) {
            //处理请求
        } else {
            //转发请求
            this.successor.handleRequest(request);
        }
    }
}
```

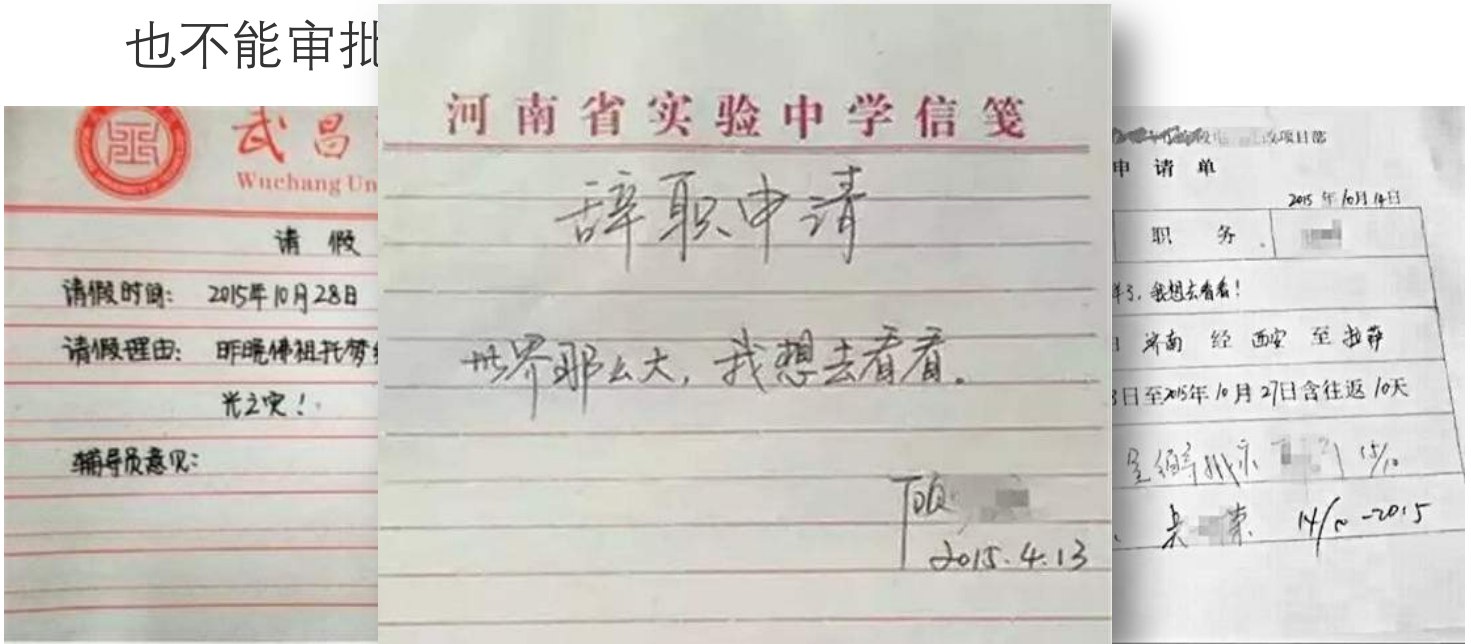
职责链模式

- 模式分析
  - 对象由每一个对象对其下家的引用连接起来形成一条链
  - 请求在链上传递，直到链上的某个对象处理此请求为止
  - 发出这个请求的客户端并不知道链上的哪一个对象最终处理这个请求，这使得系统可以在不影响客户端的情况下动态地重新组织链和分配责任



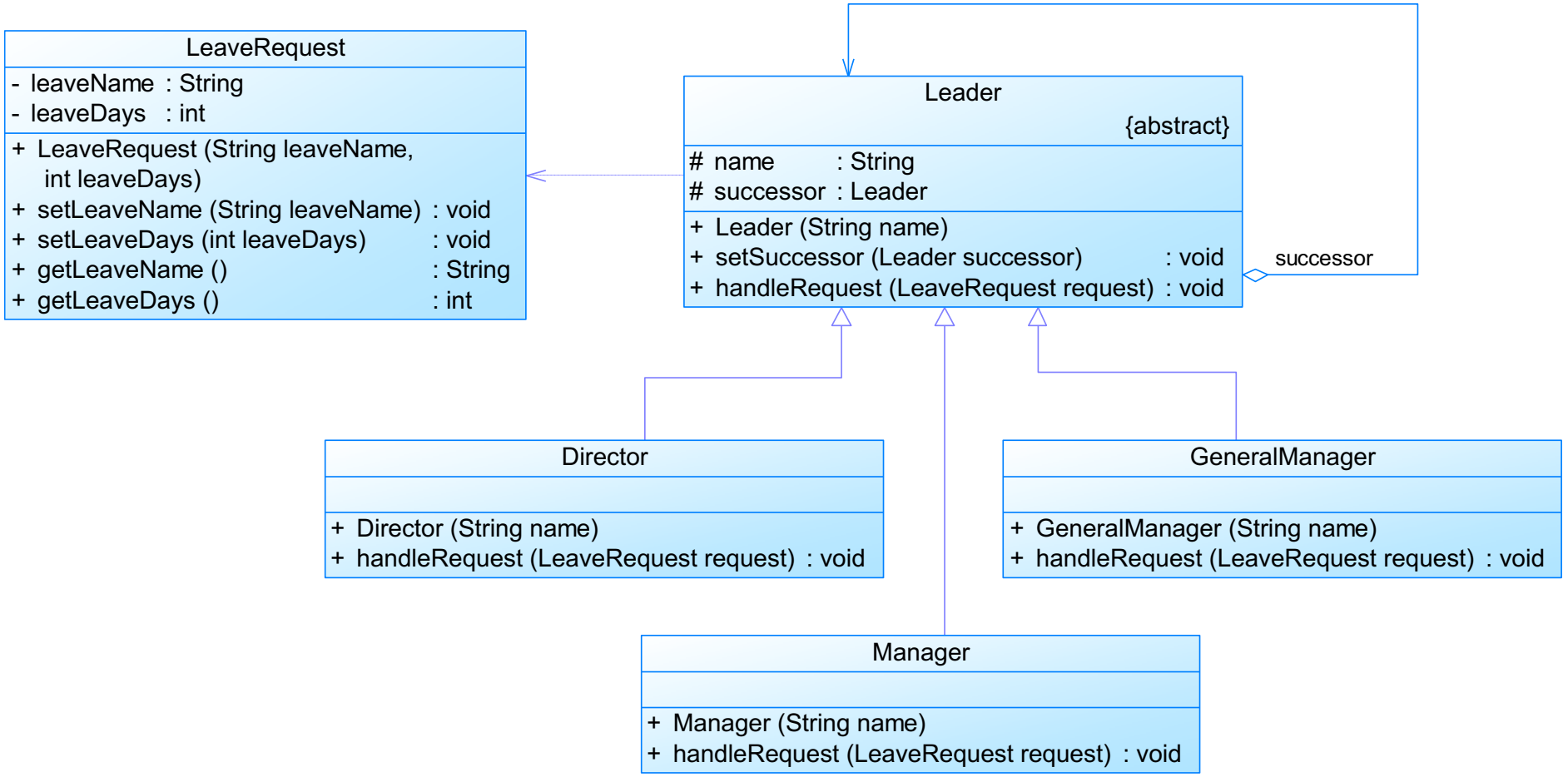
职责链模式

- 职责链模式实例与解析
  - 实例：审批请假条
    - 某OA系统需要提供一个假条审批的模块，如果员工请假天数小于3天，主任可以审批该假条；如果员工请假天数大于等于3天，小于10天，经理可以审批；如果员工请假天数大于等于10天，小于30天，总经理可以审批；如果超过30天，总经理也不能审批



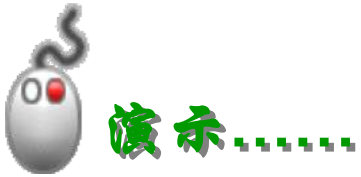
职责链模式

- 职责链模式实例与解析
  - 实例：审批假条



职责链模式

- 职责链模式实例与解析
  - 实例：审批假条
    - 参考代码 （chain_of_responsibility）



职责链模式

- 模式优缺点
  - 职责链模式的优点
    - 降低耦合度
    - 可简化对象的相互连接
    - 增强给对象指派职责的灵活性
    - 增加新的请求处理类很方便







职责链模式

- 模式优缺点
  - 职责链模式的缺点
    - 不能保证请求一定被接收
    - 系统性能将受到一定影响，而且在进行代码调试时不太方
    - 可能会造成循（tī）环（pí）调（qiu）用







职责链模式

模式适用环境



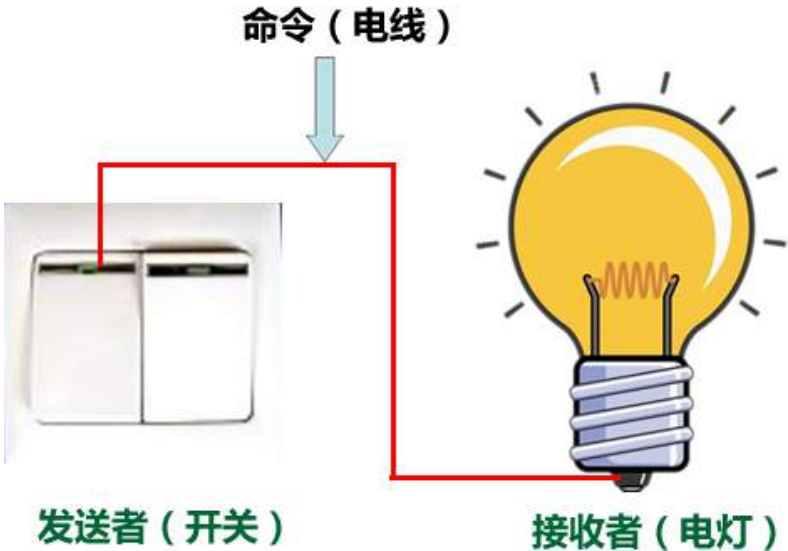
职责链模式

- 模式应用
  - Java中的异常处理机制

```
try {
      .....
} catch (ArrayIndexOutOfBoundsException e1) {
      .....
} catch (ArithmeticException e2) {
      .....
} catch (IOException e3) {
      .....
} finally {
      .....
}
```

行为型模式

- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）



命令模式



需要向某些对象发送请求，但
是谁，也不知道被请求的操作
运行时指定
令模式来进
发送者与
的调用关

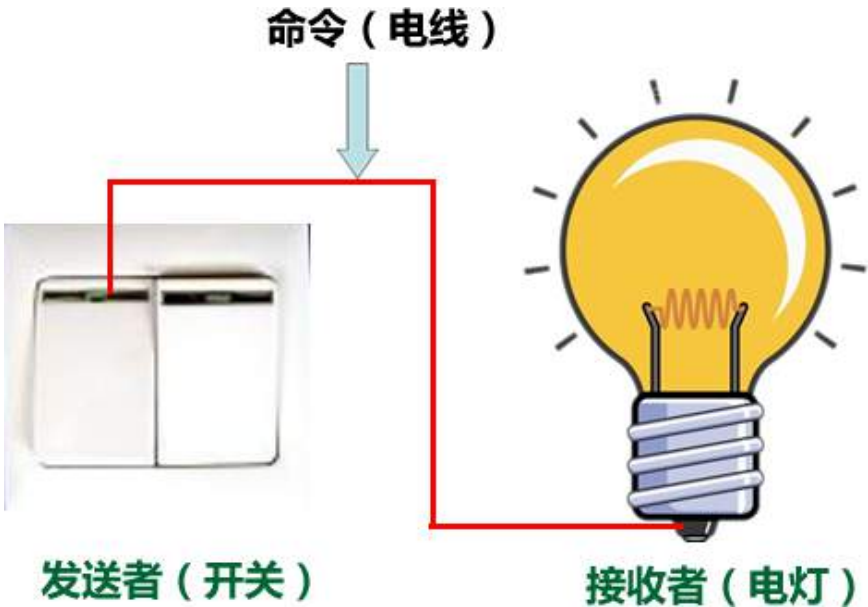




命令模式

- 模式动机
  - 命令模式可以对发送者和接收者完全解耦，发送者与接收者之间没有直接引用关系，发送请求的对象只需要知道如何发送请求，而不必知道如何完成请求。这就是命令模式的模式动机





命令模式

- 模式定义
  - 命令模式（Command Pattern）将一个请求封装为一个对象，从而使得可以根据请求参数对客户进行分类，对不同的请求进行排队，记录请求日志，以及支持可撤销的操作。命令模式是一种对象行为型模式

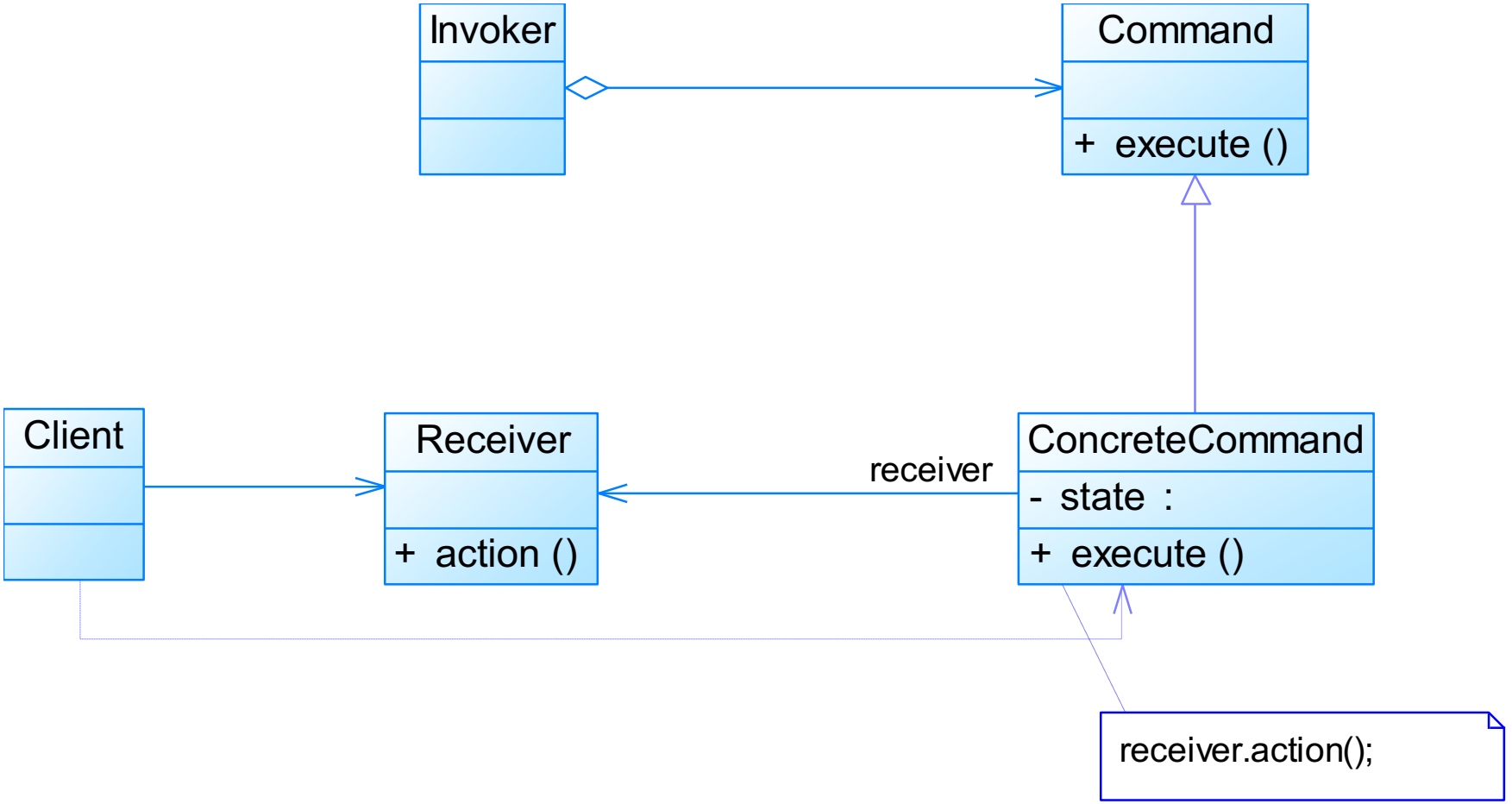
对事不对人

命令模式

命令模式包含如下角色：

- Command: 抽象命令类
- ConcreteCommand: 具体命令类
- Invoker: 调用者
- Receiver: 接收者
- Client:客户类

模式结构



命令模式

- 模式分析
  - 命令模式的本质是对命令进行封装，将发出命令的责任和执行命令的责任分割开
  - 每一个命令都是一个操作：请求的一方发出请求，要求执行一个操作；接收的一方收到请求，并执行操作
  - 命令模式允许请求的一方和接收的一方独立开来，使得请求的一方不必知道接收请求的一方的接口，更不必知道请求是怎么被接收，以及操作是否被执行、何时被执行，以及是怎么被执行的

请派人落实此事！

命令模式

- 模式分析
  - 典型的抽象命令类代码：

```
public abstract class Command {
     public abstract void execute();
}
```

命令模式

- 模式分析
  - 典型的调用者代码：

```
public class Invoker {
    private Command command;
    public Invoker(Command command) {
        this.command = command;
    }
    public void setCommand(Command command) {
        this.command = command;
    }
    //业务方法，用于调用命令类的方法
    public void call() {
        command.execute();
    }
}
```

命令模式

- 模式分析
  - 典型的具体命令类代码：

```
public class ConcreteCommand extends Command {
    private Receiver receiver;
    public void execute() {
        receiver.action();
    }
}
```

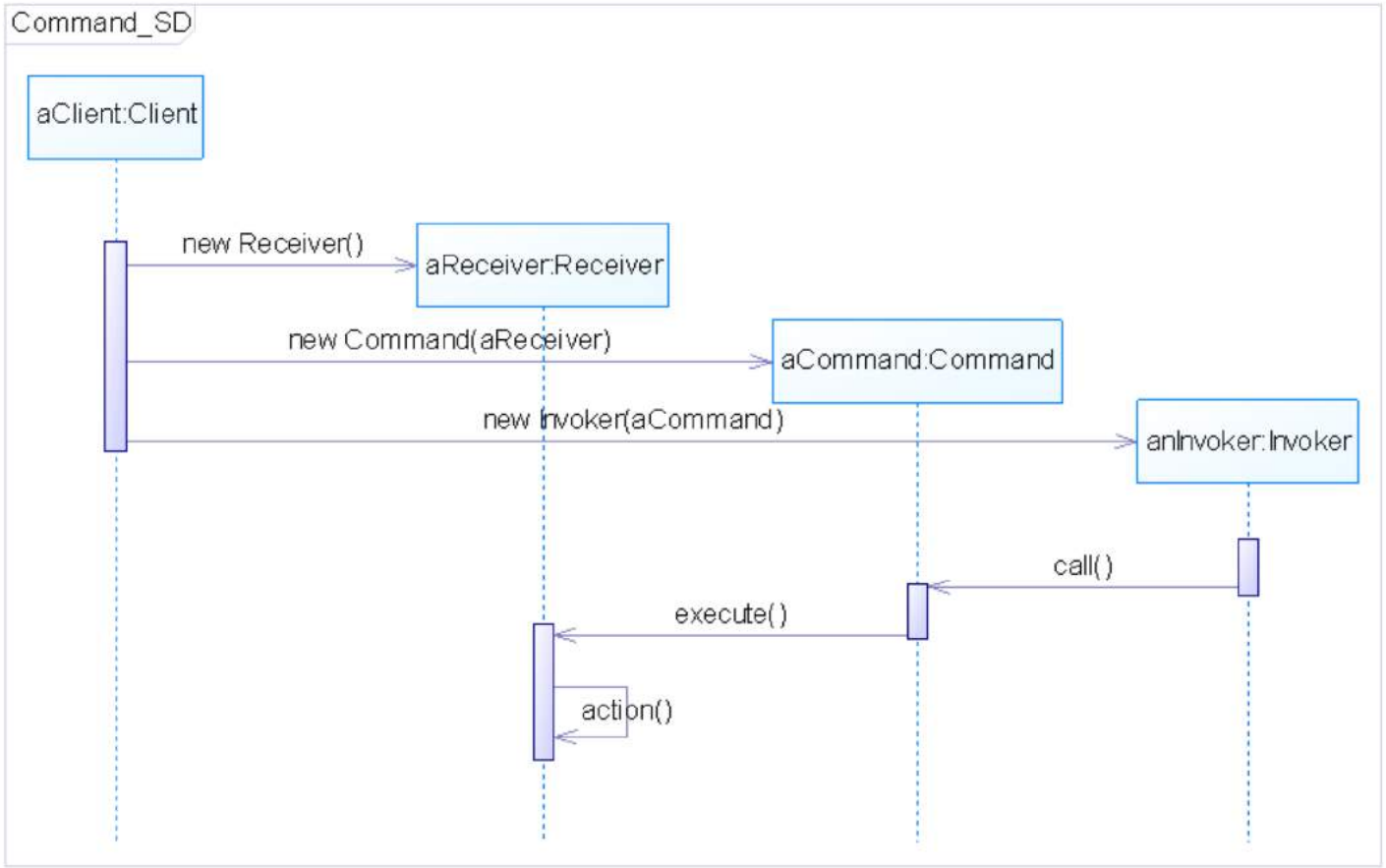
命令模式

- 模式分析
  - 典型的请求接收者代码：

```
public class Receiver {
     public void action() {
          //具体操作
     }
}
```

命令模式

- 模式分析
  - 命令模式顺序图 ：

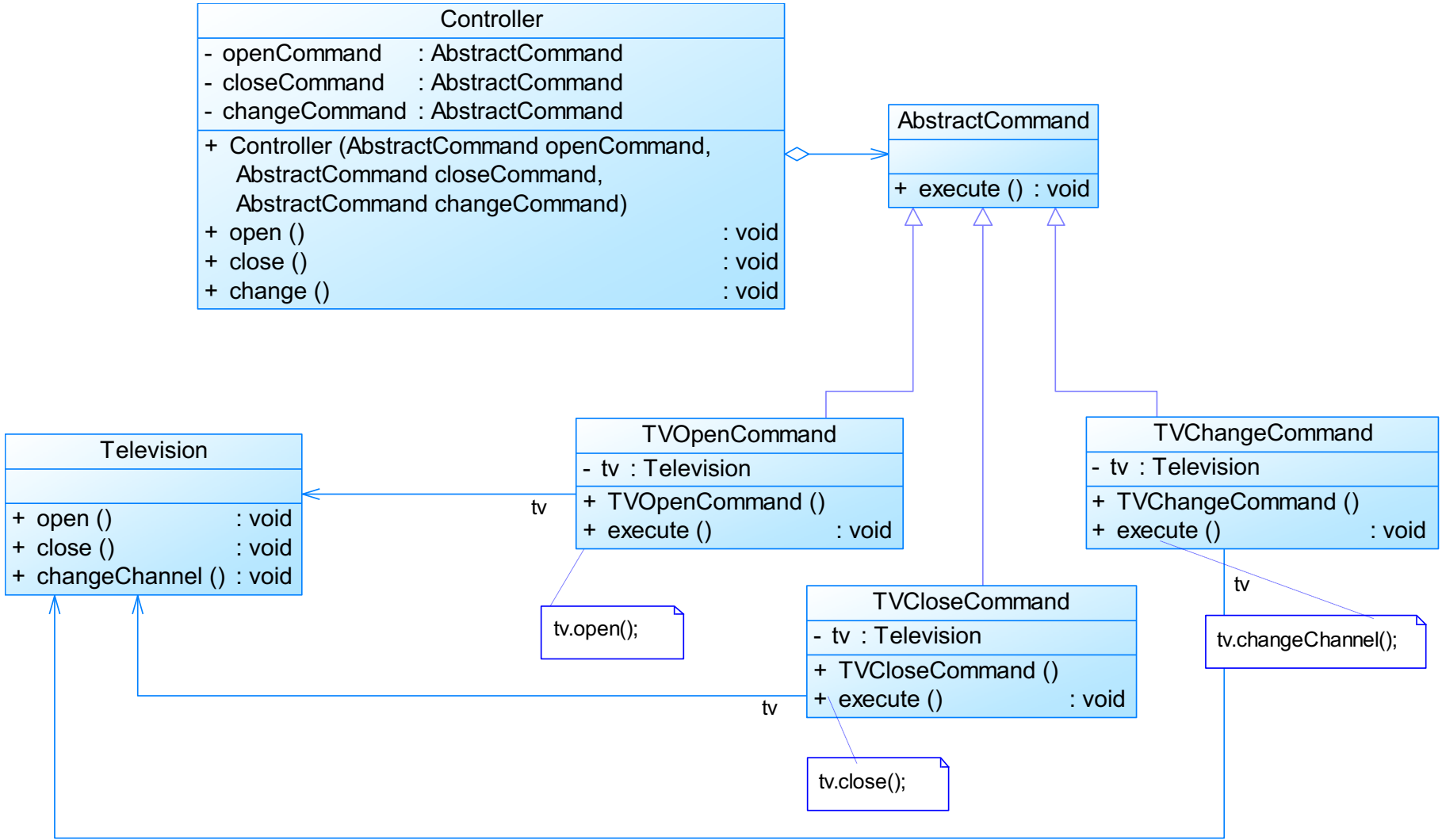


命令模式

- 命令模式实例与解析
  - 实例：电视机遥控器
    - 电视机是请求的接收者，遥控器是请求的发送者，遥控器上有一些按钮，不同的按钮对应电视机的不同操作。抽象命令角色由一个命令接口来扮演，有三个具体的命令类实现了抽象命令接口，这三个具体命令类分别代表三种操作：打开电视机、关闭电视机和切换频道。显然，电视机遥控器就是一个典型的命令模式应用实例。

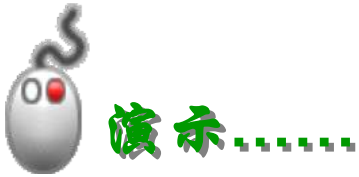
命令模式

命令模式实例与解析



命令模式

- 命令模式实例与解析
  - 实例：电视机遥控器
    - 参考代码（command）



命令模式

- 模式优缺点
  - 命令模式的优点
    - 降低系统的耦合度
    - 新的命令可以很容易地加入到系统中
    - 可以比较容易地设计一个命令队列和宏命令（组合命令）

命令模式

- 模式优缺点
  - 命令模式的缺点
    - 使用命令模式可能会导致某些系统有过多的具体命令类。因为针对每一个命令都需要设计一个具体命令类，因此某些系统可能需要大量具体命令类，这将影响命令模式的使用。



厦大软件体系结构课观光团：
你得先这样，然后再这样，然后再这样，最后再这样。。。

命令模式

- 模式适用环境
  - 在以下情况下可以使用命令模式：
    - 系统需要将请求调用者和请求接收者解耦，使得调用者和接收者不直接交互。
    - 系统需要在不同的时间指定请求、将请求排队和执行请求。
    - 系统需要支持命令的撤销(Undo)操作和恢复(Redo)操作。
    - 系统需要将一组操作组合在一起，即支持宏命令。

命令模式

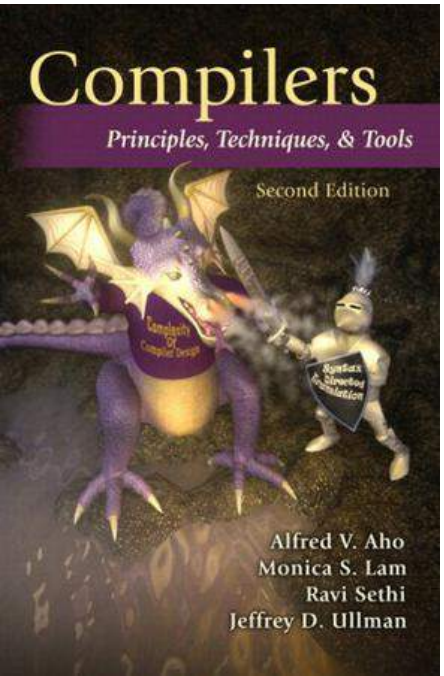
- 模式应用
  - Java语言使用命令模式实现AWT/Swing GUI的委派事件模型 (Delegation Event Model, DEM)
    - 在AWT/Swing中，Frame、Button等界面组件是请求发送者，而AWT提供的事件监听器接口和事件适配器类是抽象命令接口，用户可以自己写抽象命令接口的子类来实现事件处理，即实现具体命令类，而在具体命令类中可以调用业务处理方法来实现该事件的处理。对于界面组件而言，只需要了解命令接口即可，无须关心接口的实现，组件类并不关心实际操作，而操作由用户来实现。

命令模式

- 模式分析
  - 命令模式使请求本身成为一个对象，这个对象和其他对象一样可以被存储和传递
  - 命令模式的关键在于引入了抽象命令接口，且发送者针对抽象命令接口编程，只有实现了抽象命令接口的具体命令才能与接收者相关联

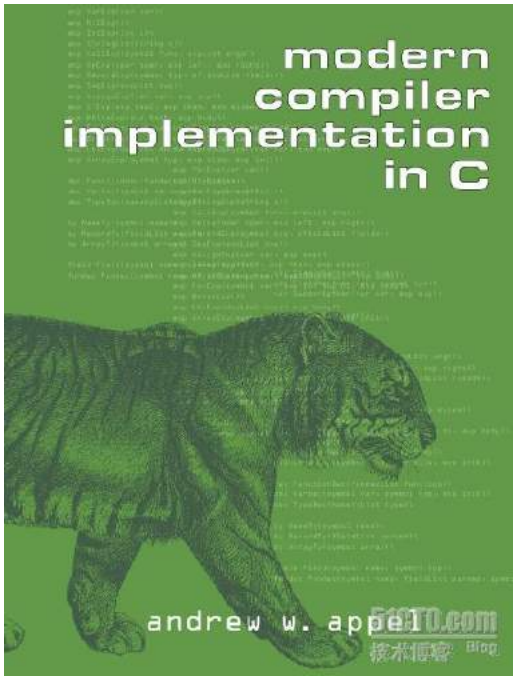
第10章

编译原理

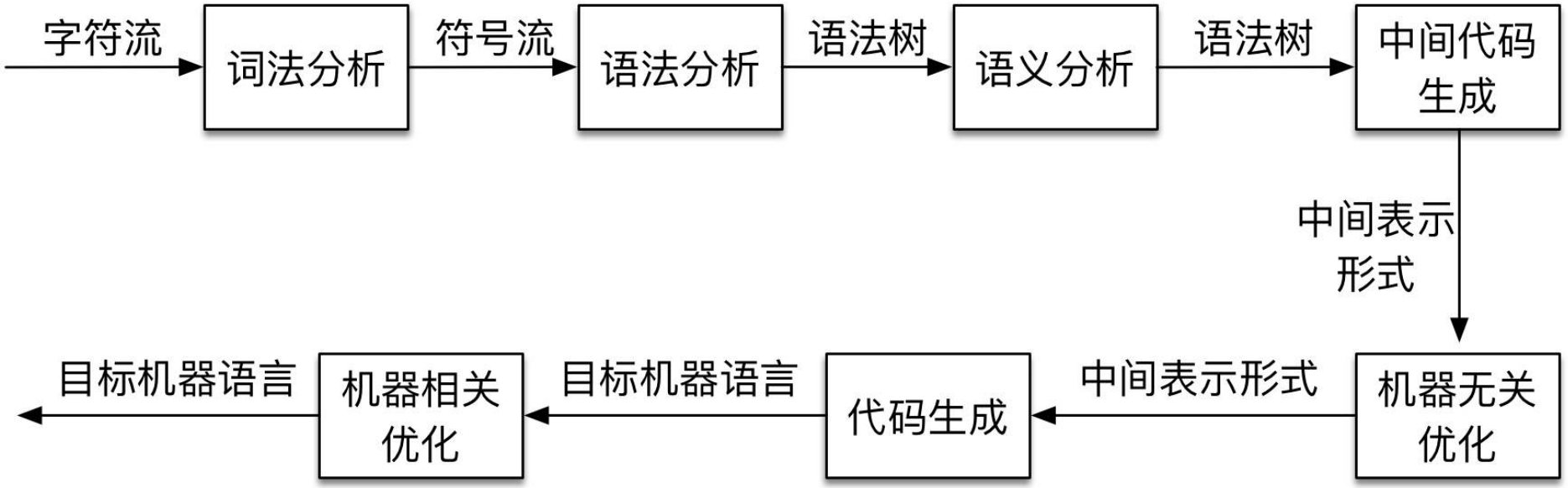


陈龙彪

厦门大学信息学院
2019年4月



教学大纲



- 考试形式：闭卷，手写编译器，C/C++/Java任选
- 考试时间：一天

行为型模式

- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）

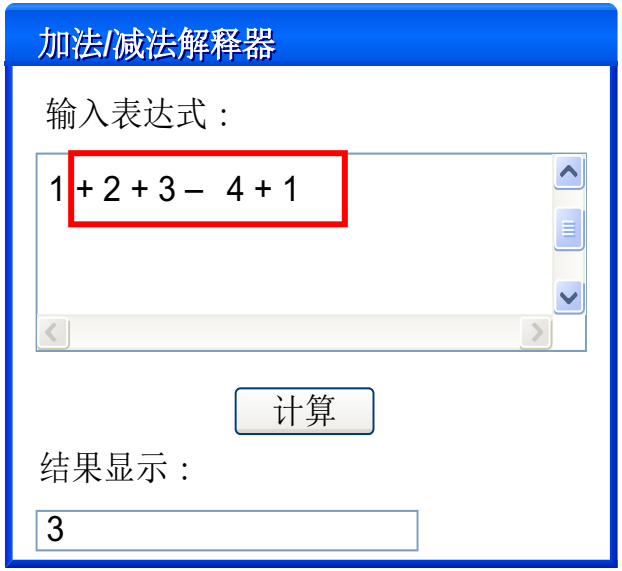




要不要弄个新的编译器玩玩？

解释器模式

- 模式动机
  - 如果在系统中某一特定类型的问题发生的频率很高，可以考虑将这些问题的实例表述为一个语言中的句子，从而构建一个解释器，通过解释这些句子来解决这些问题
  - 解释器模式描述了如何构成一个简单的语言解释器，主要应用在使用面向对象语言开发的编译器中



解释器模式

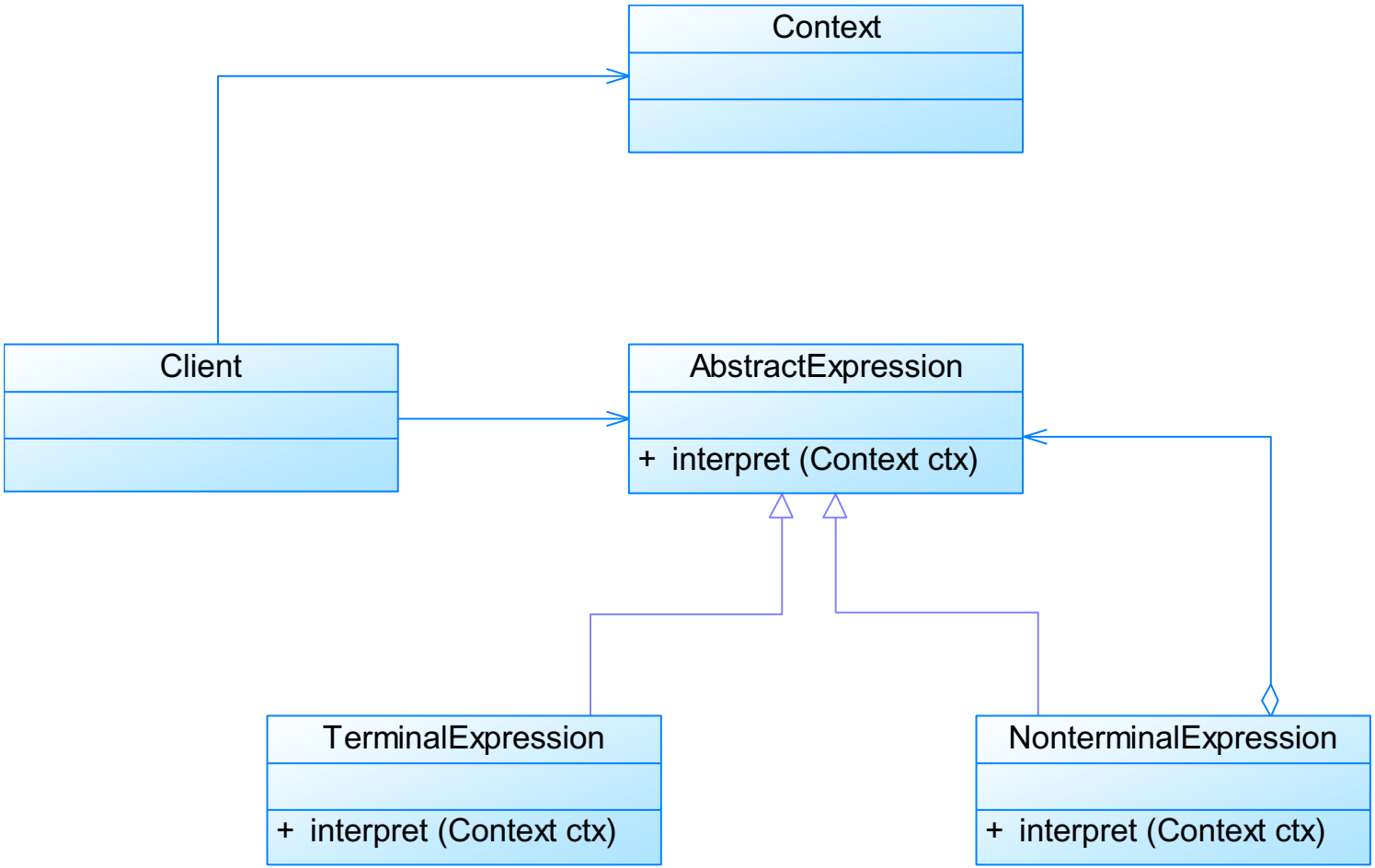
- 模式定义
  - 解释器模式（Interpreter Pattern）：定义语言的文法，并且建立一个解释器来解释该语言中的句子，这里的“语言”意思是使用规定格式和语法的代码，它是一种类行为型模式

解释器模式

解释器模式包含如下角色：

- AbstractExpression: 抽象表达式
- TerminalExpression: 终结符表达式
- NonterminalExpression: 非终结符表达式
- Context: 环境类
- Client: 客户类

模式结构



解释器模式

- 模式分析
  - 解释器模式描述了
    何在该语言中表示

Soufflé, 舒芙蕾, 蛋奶酥

厦大软件

主題　　是

A は B です

- わたし は 王
- 陳さん は 学生
- 田中さん は 日本人

名詞　　名詞







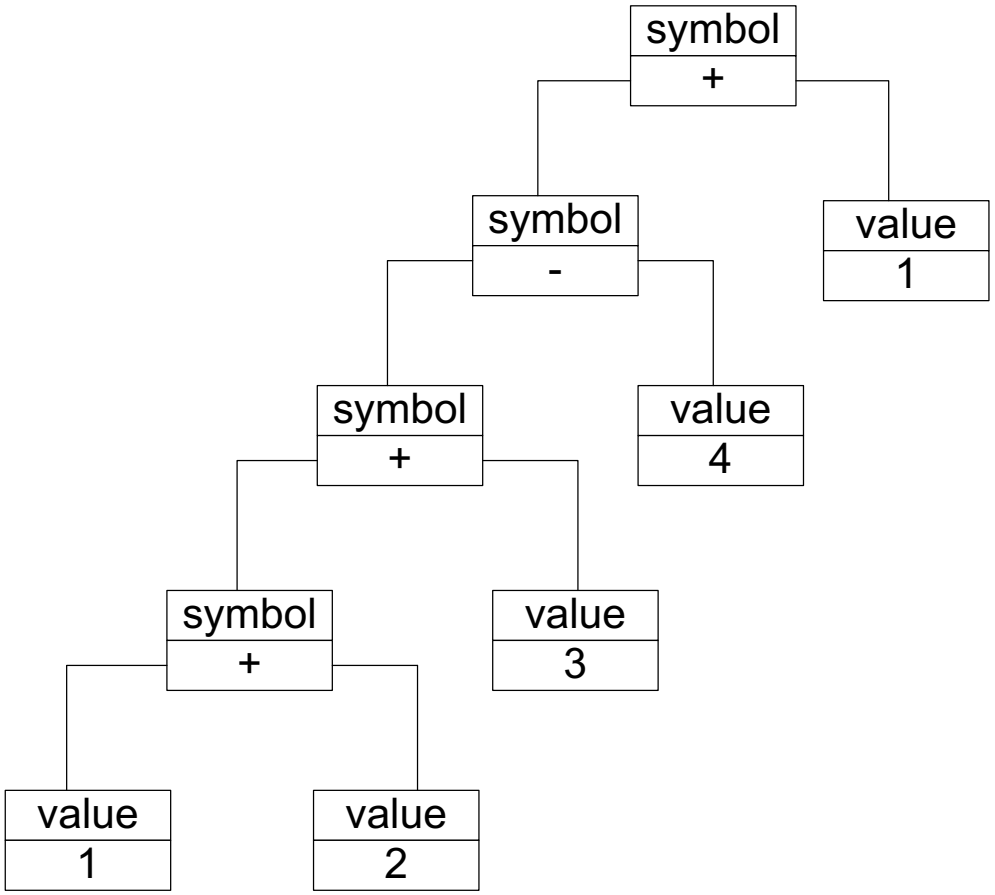
解释器模式

- 模式分析
  - 文法规则实例：
    - expression ::= value l symbol
    - symbol ::= expression '+' expression l expression '-' expression
    - value ::= an integer //一个整数值
  - 在文法规则定义中可以使用一些符号来表示不同的含义，如使用“l”表示或，使用“{”和“}”表示组合，使用“*”表示出现0次或多次等

解释器模式

模式分析

- 抽象语法树（Abstract Syntax Tree，AST）可以直观地表示语言的构成，每一棵抽象语法树对应一个语言实例



解释器模式

- 模式分析
  - 抽象语法树描述了如何构成一个复杂的句子
  - 通过对抽象语法树的分析，可以识别出语言中的终结符和非终结符类
  - 在解释器模式中，每一种终结符和非终结符都有一个具体类与之对应
  - 正因为使用类来表示每一个语法规则，使得系统具有较好的扩展性和灵活性

解释器模式

- 模式分析
  - 典型的抽象表达式类实现代码：

```
public abstract class AbstractExpression {
    public abstract void interpret(Context ctx);
}
```

解释器模式

- 模式分析
  - 典型的终结符表达式类实现代码：

```
public class TerminalExpression extends AbstractExpression {
    public void interpret(Context ctx) {
        //对于终结符表达式的解释操作
    }
}
```

解释器模式

- 模式分析
  - 典型的非终结符表达式类实现代码：

```
public class NonterminalExpression extends AbstractExpression {
     private AbstractExpression left;
     private AbstractExpression right;

     public NonterminalExpression(AbstractExpression left,
AbstractExpression right) {
          this.left = left;
          this.right = right;
     }
     public void interpret(Context ctx) {
          //递归调用每一个组成部分的 interpret()方法
          //在递归调用时指定组成部分的连接方式，即非终结符的功能
     }
}
```

解释器模式

- 模式分析
  - 典型的环境类实现代码：

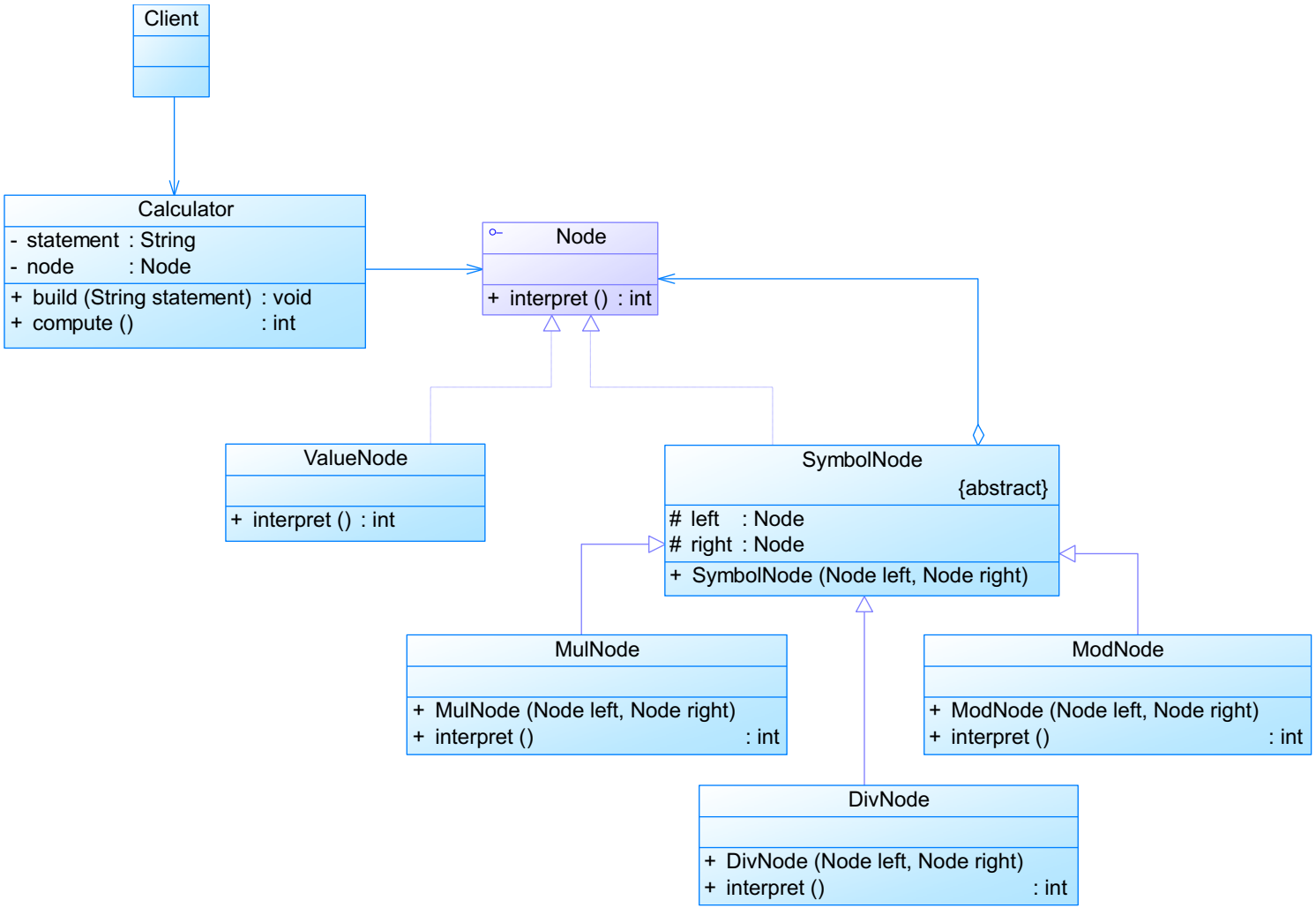
```
public class Context {
    private HashMap map = new HashMap();
    public void assign(String key, String value) {
        //往环境类中设值
    }
    public String lookup(String key) {
        //获取存储在环境类中的值
    }
}
```

解释器模式

- 解释器模式实例与解析
  - 实例：数学运算解释器
    - 现需要构造一个语言解释器，使得系统可以执行整数间的乘、除和求模运算。如用户输入表达式“3 * 4 / 2 % 4”，输出结果为2。使用解释器模式实现该功能。

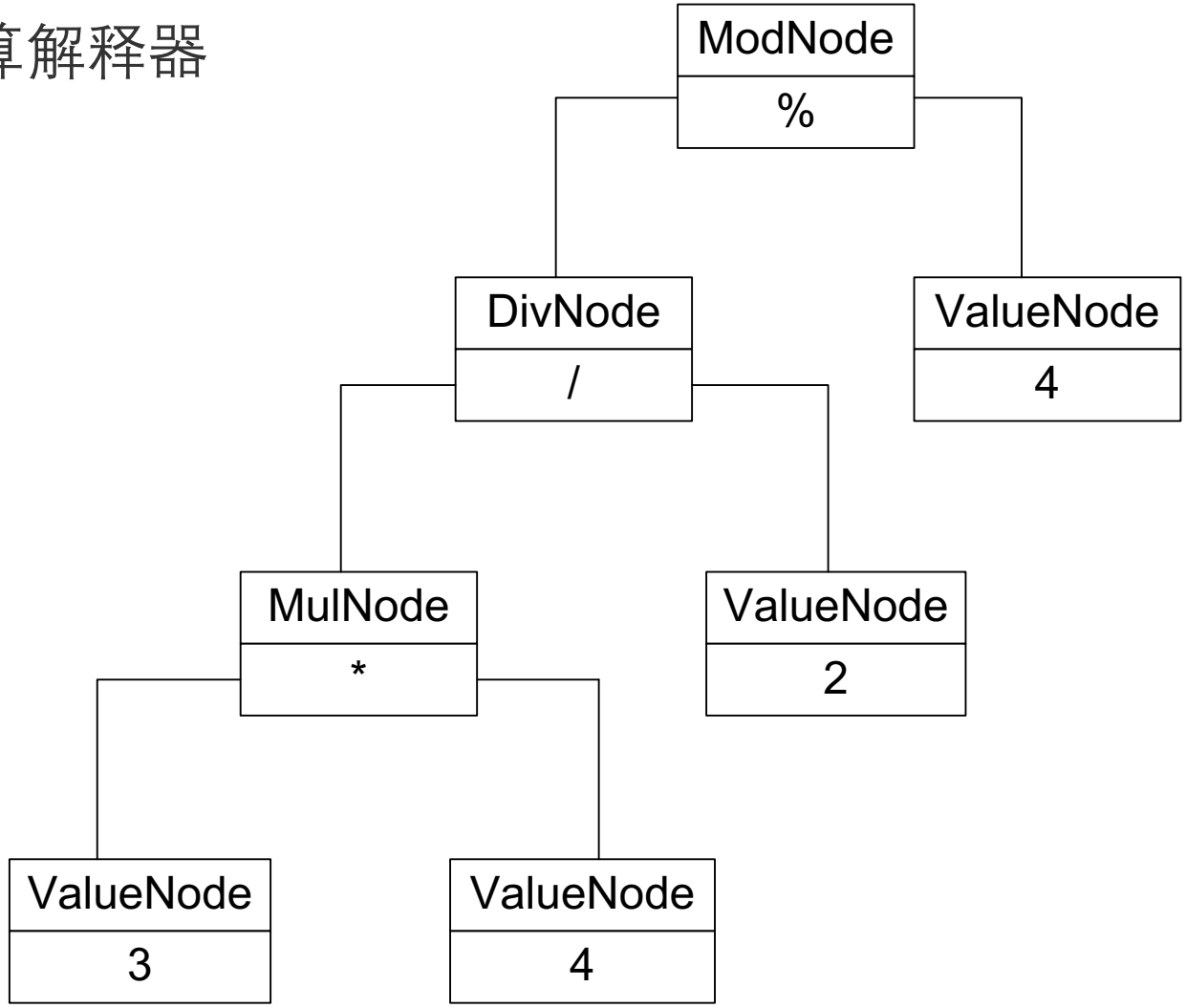
解释器模式

解释器模式实例与解析



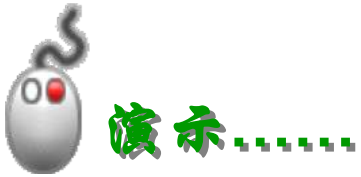
解释器模式

- 解释器模式实例与解析
  - 实例：数学运算解释器



解释器模式

- 解释器模式实例与解析
  - 实例：数学运算解释器
    - 参考代码 （interpreter）



解释器模式

- 解释器模式实例与解析
  - 实例：埃拉托斯芬筛法@中文编程语言

https://github.com/audreyt/lingua-sinica-perlyuyan

```
Input : n =10
Output : 2 3 5 7

Input : n = 20
Output: 2 3 5 7 11 13 17 19
```

```
for (int p=2; p*p<=n; p++)
{
    // If prime[p] is not change
    if (prime[p] == true)
    {
        // Update all multiples
        // equal to the square o
        // numbers which are mul
        // less than p^2 are alr
        for (int i=p*p; i<=n; i
            prime[i] = false;
    }
}
```

唐凤的 PerlYuYan 堪称最优雅的**中文**编程语言。
其实准确地说它是一个 Perl module，让每个汉字对应一个符号或关键字。
于是下面这是埃氏筛法的代码：

```
# The Sieve of Eratosthenes - 埃拉托斯芬篩法
use Lingua::Sinica::PerlYuYan;

   用籌兮用嚴。井涸兮無礙
。印曰最高矣　又道數然哉。
。截起吾純風　賦小入大合。
。習予吾陣地　並二至純風。
。當起段賦取　加陣地合始。
。陣地賦篩始　繫繫此雜段。
。終陣地兮印　正道次標哉。
。輸空接段點　列終註泰來。

use integer; use strict; # 涸; 無礙;
print '最高', "數:";
chomp (my $z = );
study unshift my @w, 2 .. $z;
while ($/ = shift +@w) {;
@w = grep { ~~ $_ % $/;
} @w; print +"next :";
printf " ".$/."\n" } # 泰來;
```

解释器模式

- 模式优缺点
  - 解释器模式的优点
    - 易于改变和扩展文法
    - 易于实现文法

解释器模式

- 模式优缺点
  - 解释器模式的缺点
    - 对于复杂文法难以维护
    - 执行效率较低
    - 应用场景很有限

解释器模式

- 模式适用环境
  - 在以下情况下可以使用解释器模式：
    - 可以将一个需要解释执行的语言中的句子表示为抽象语法树
    - 一些重复出现的问题可以用一种简单的语言来进行表达
    - 文法较为简单
    - 效率不是关键问题

解释器模式

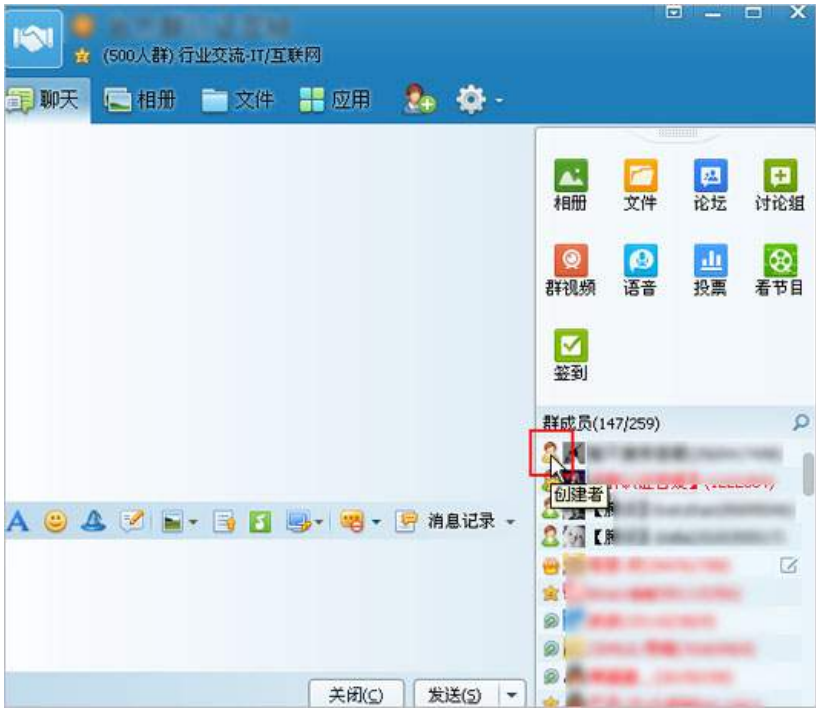
- 模式应用
  - 解释器模式在使用面向对象语言实现的编译器中得到了广泛的应用，如Smalltalk语言的编译器
  - 目前有一些基于Java抽象语法树的源代码处理工具，如在Eclipse中就提供了Eclipse AST，它是Eclipse JDT的一个重要组成部分，用来表示Java语言的语法结构，用户可以通过扩展其功能，创建自己的文法规则

解释器模式

- 模式扩展
  - 数学表达式解析器简介
    - 在实际项目开发中如果需要解析数学公式，无须再运用解释器模式进行设计，可以直接使用一些第三方解析工具包，它们可以统称为数学表达式解析器(Math Expression Parser, MEP)，如Expression4J、Jep、JbcParser、Symja、Math Expression String Parser(MESP)等来取代解释器模式，它们可以方便地解释一些较为复杂的文法，功能强大，且使用简单，效率较好

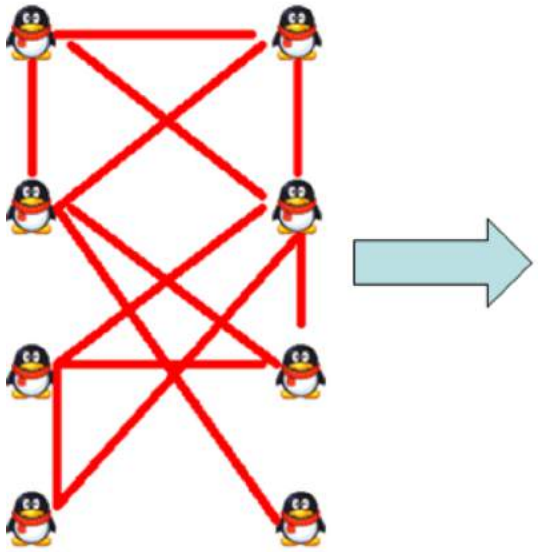
行为型模式

- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）



中介者模式

模式动机




友情万岁(12345678)
(普通群) 生活休闲
交谈中请勿轻信汇款、中奖信息、陌生电话，勿使用外挂软件。
群动态
你的群相册最近没有新照片
立刻上传
群成员(1/7)
搜索群成员
赵六
李四
王五
小强
小莉
小花
张三
消息记录
猛点下载，高手实盘数据抢先看
关闭(C)
发送(S)

中介者模式

- 模式动机
  - 在用户与用户直接聊天的设计方案中，用户对象之间存在很强的关联性，将导致系统出现如下问题：
    - 系统结构复杂：对象之间存在大量的相互关联和调用，若有一个对象发生变化，则需要跟踪和该对象关联的其他所有对象
    - 对象可重用性差：由于一个对象和其他对象具有很强的关联，这些对象更像一个不可分割的整体，职责较为混乱。
    - 系统扩展性低：增加一个新的对象需要在原有相关对象上增加引用，系统耦合度很高，对象操作很不灵活，扩展性差

中介者模式

- 模式动机
  - 在面向对象的软件设计中，根据“单一职责原则”，应该尽量将对象细化，使其只负责或呈现单一职责
  - 对于一个模块，可能由很多对象构成，而且这些对象之间可能存在相互的引用，为了减少对象两两之间复杂的引用关系，使之成为一个松耦合的系统，我们需要使用中介者模式，这就是中介者模式的模式动机

中介者模式

- 模式定义
  - 中介者模式（Mediator Pattern）定义：用一个中介对象来封装一系列的对象交互，中介者使各对象不需要显式地相互引用，从而使其耦合松散，而且可以独立地改变它们之间的交互
  - 中介者模式又称为调停者模式，是一种对象行为型模式



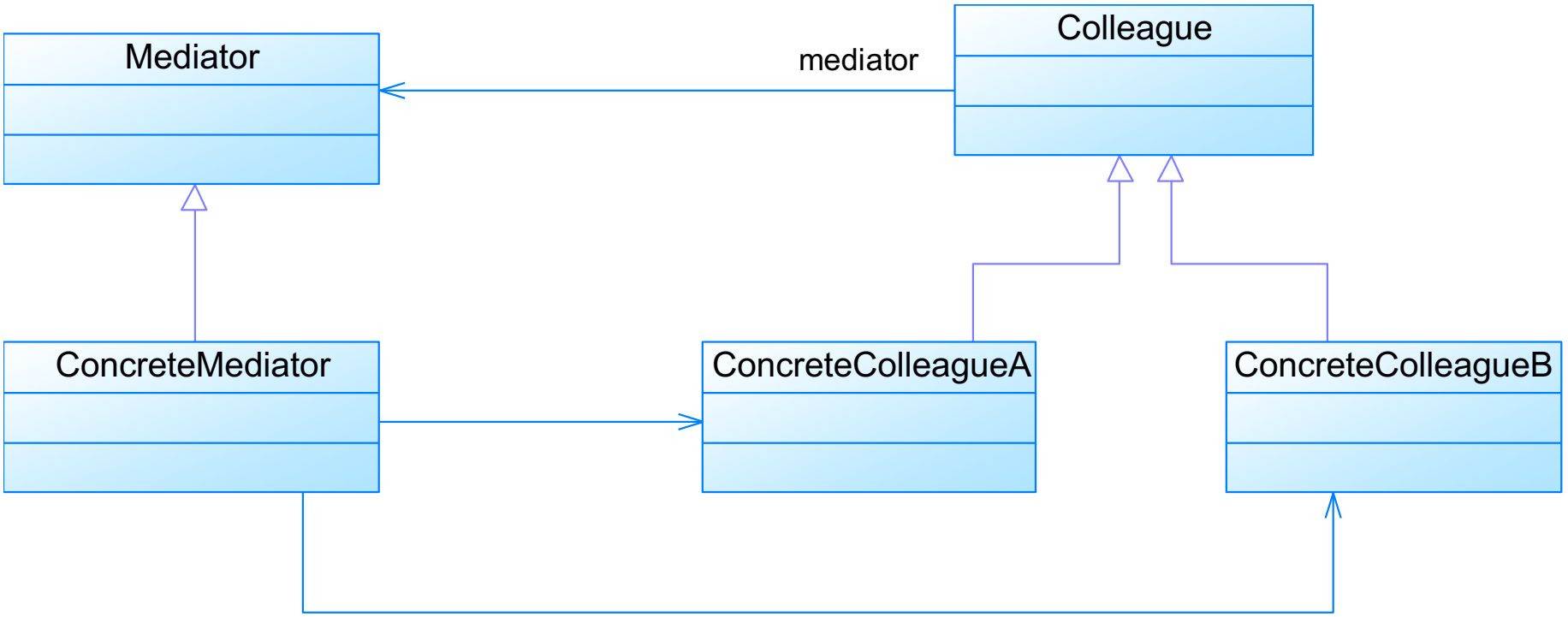


中介者模式

中介者模式包含如下角色：

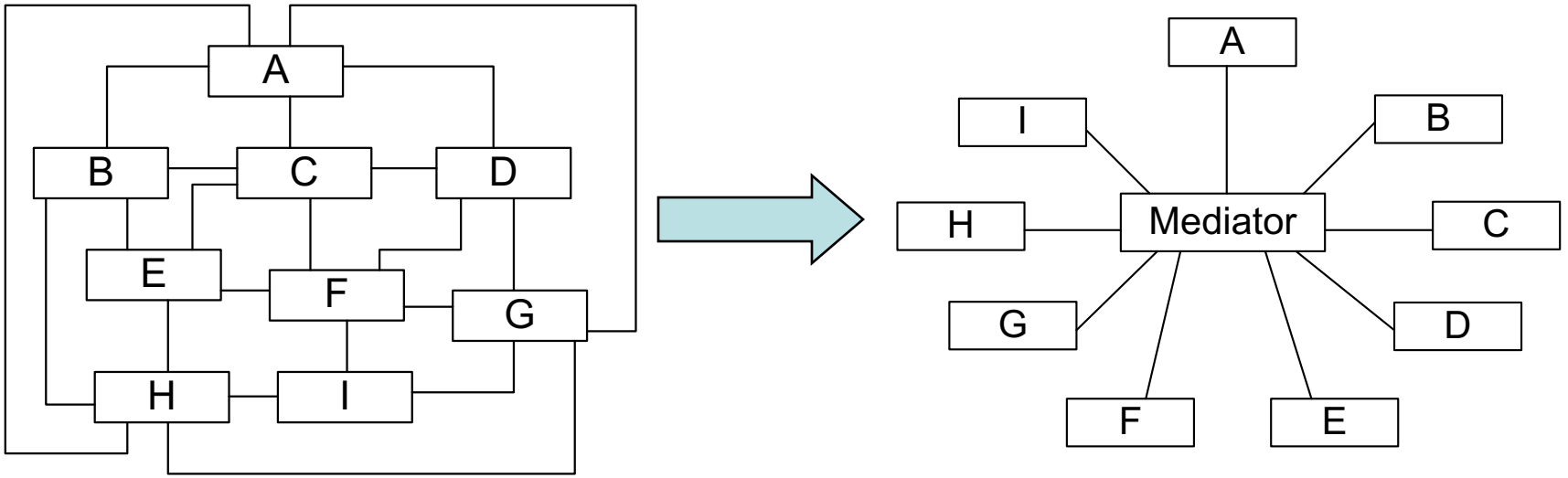
- Mediator: 抽象中介者
- ConcreteMediator: 具体中介者
- Colleague: 抽象同事类
- ConcreteColleague: 具体同事类

模式结构



中介者模式

- 模式分析
  - 中介者模式可以使对象之间的关系数量急剧减少



中介者模式

- 模式分析
  - 中介者承担两方面的职责：
    - 中转作用（结构性）：通过中介者提供的中转作用，各个同事对象就不再需要显式引用其他同事，当需要和其他同事进行通信时，通过中介者即可
    - 协调作用（行为性）：中介者可以更进一步的对同事之间的关系进行封装，同事可以一致地和中介者进行交互，而不需要指明中介者需要具体怎么做，中介者根据封装在自身内部的协调逻辑，对同事的请求进行进一步处理，将同事成员之间的关系行为进行分离和封装

中介者模式

- 模式分析
  - 典型的抽象中介者类代码：

```
public abstract class Mediator {
     protected ArrayList colleagues;
     public void register(Colleague colleague) {
          colleagues.add(colleague);
     }

     public abstract void operation();
}
```

中介者模式

- 模式分析
  - 典型的具体中介者类代码：

```
public class ConcreteMediator extends Mediator {
     public void operation() {
          ......
          ((Colleague)(colleagues.get(0))).method1();
          ......
     }
}
```

中介者模式

- 模式分析
  - 典型的抽象同事类代码：

```
public abstract class Colleague {
    protected Mediator mediator;

    public Colleague(Mediator mediator) {
        this.mediator = mediator;
    }

    public abstract void method1();

    public abstract void method2();
}
```

中介者模式

- 模式分析
  - 典型的具体同事类代码：

```
public class ConcreteColleague extends Colleague {
    public ConcreteColleague(Mediator mediator) {
        super(mediator);
    }

    public void method1() {
        ......
    }

    public void method2() {
        mediator.operation1();
    }
}
```

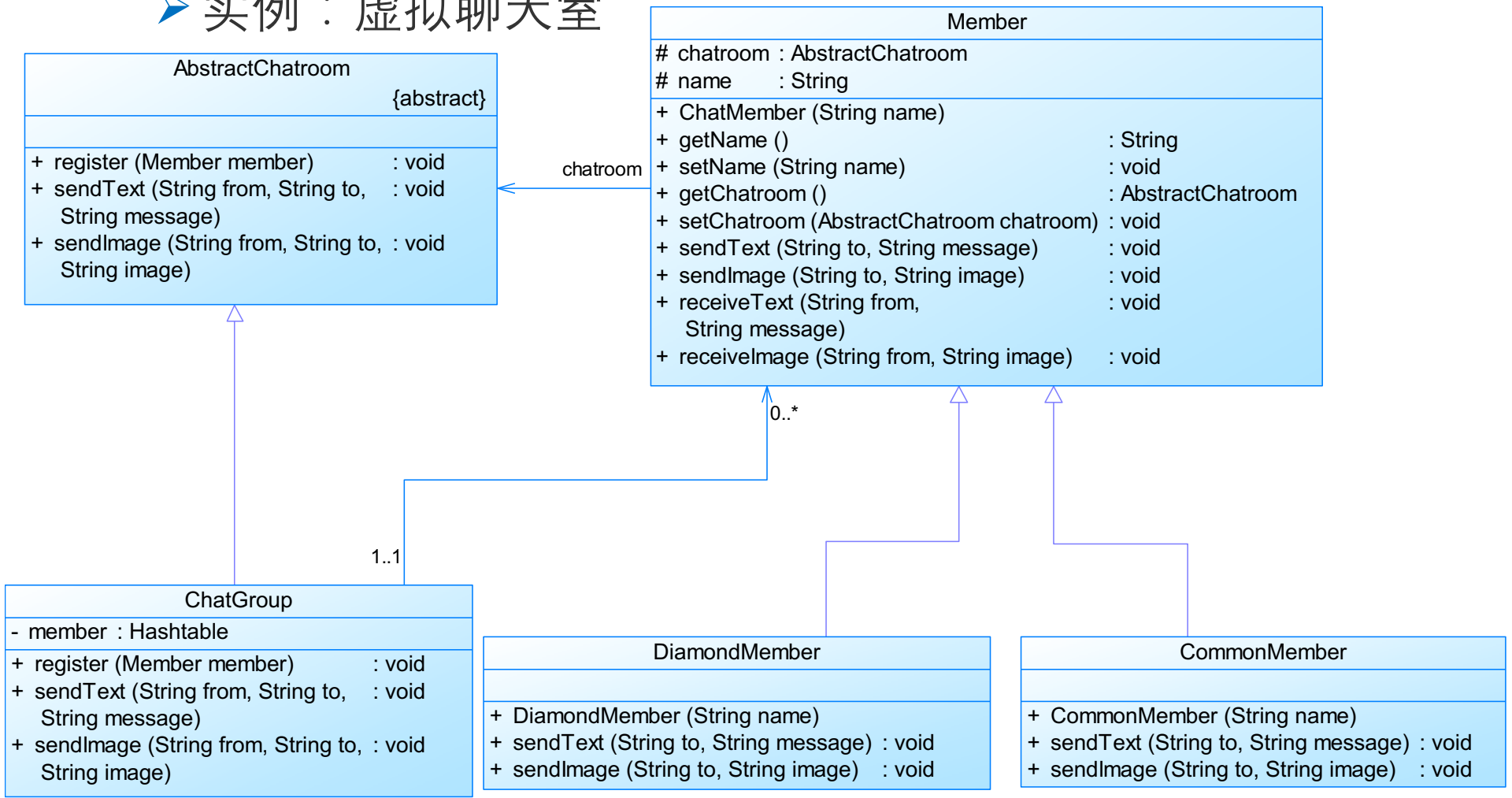
中介者模式

- 中介者模式实例与解析
  - 实例：泰坦尼克号聊天室
    - 泰坦尼克号欲增加一个聊天室，允许乘客通过该聊天室交流：
      普通会员（CommonMember）只能给其他普通会员发送信息，
      钻石会员（DiamondMember）则可以给所有会员发送信息
    - 该聊天室可以对隐私字符进行过滤，如乘客的船舱号码
    - 请用中介者模式设计该虚拟聊天室

中介者模式

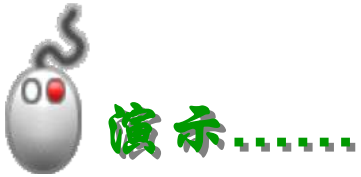
中介者模式实例与解析

实例：虚拟聊天室



中介者模式

- 中介者模式实例与解析
  - 实例：虚拟聊天室
    - 参考代码 （mediator）



中介者模式

- 模式优缺点
  - 中介者模式的优点
    - 简化了对象之间的交互
    - 将各同事解耦
    - 减少子类生成
    - 可以简化各同事类的设计和实现

中介者模式

- 模式优缺点
  - 中介者模式的缺点
    - 在具体中介者类中包含了同事之间的交互细节，可能会导致具体中介者类非常复杂，使得系统难以维护

中介者模式

- 模式适用环境
  - 在以下情况下可以使用中介者模式：
    - 系统中对象之间存在复杂的引用关系，产生的相互依赖关系结构混乱且难以理解
    - 一个对象由于引用了其他很多对象并且直接和这些对象通信，导致难以复用该对象
    - 想通过一个中间类来封装多个类中的行为，而又不想生成太多的子类。可以通过引入中介者类来实现，在中介者中定义对象交互的公共行为，如果需要改变行为则可以增加新的中介者类

中介者模式

- 模式应用
  - 中介者模式在事件驱动类软件中应用比较多，在设计GUI应用程序时，组件之间可能存在较为复杂的交互关系，一个组件的改变将影响与之相关的其他组件，此时可以使用中介者模式来对组件进行协调

刘备

云长，长坂坡失守，速来
😭😭😭😭😭😭😭

主公，你现在何

刘备



主公莫哭，云长这便策马

刘备

😖 我等你！

曹操

诸葛亮，你要是跟俺干，俺
薪几十万。

孙权

你要是跟俺干，俺给你美女

刘备

先生，您还是跟俺干吧，
俺，俺，俺，俺会取消你的农

🤝 OK 一言为定，马

姜维

丞相，空城之计，神鬼莫测。不过
末将尚有一事不明。

诸葛亮

讲

姜维

丞相何以知司马懿生平谨慎，必
不弄险？

诸葛亮

因为他是金牛座。

姜维

👍

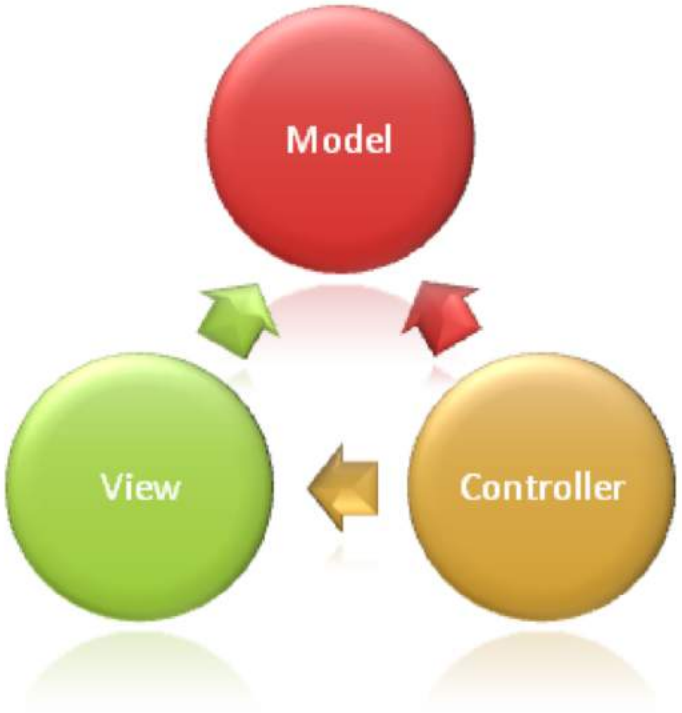
诸葛亮



【转载】12 星座性格分析
——第一星座网

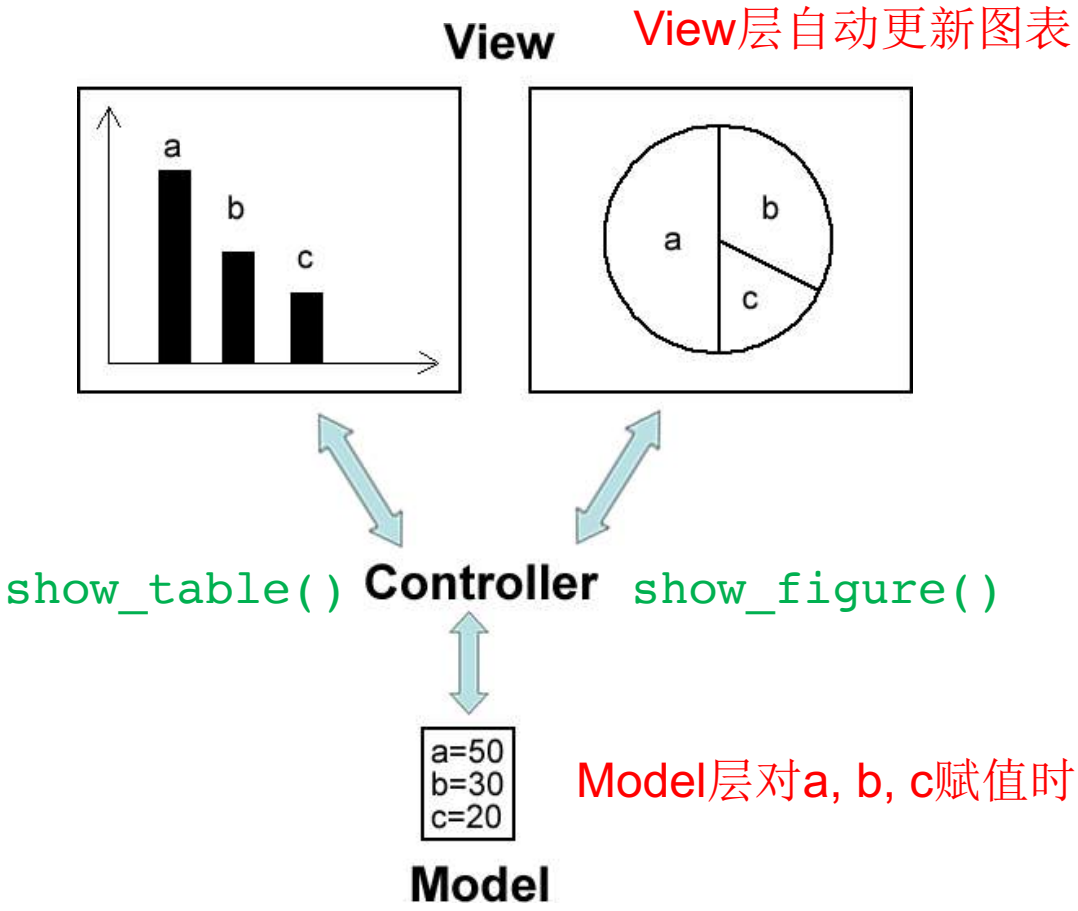
行为型模式

- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）



观察者模式

- 模式动机


View
View层自动更新图表
a
b
c
a
b
c
show_table()
Controller
show_figure()
a=50
b=30
c=20
Model层对a, b, c赋值时
Model

观察者模式

- 模式动机
  - 建立一种对象与对象之间的依赖关系，一个对象发生改变时将自动通知其他对象，其他对象将相应做出反应
    - 发生改变的对象称为观察目标
    - 而被通知的对象称为观察者
  - 一个观察目标可以对应多个观察者，而且这些观察者之间没有相互联系，可以根据需要增加和删除观察者，使得系统更易于扩展，这就是观察者模式的模式动机

观察者模式

- 模式定义
  - 观察者模式（Observer Pattern）定义了对象间的一种一对多依赖关系，使得每当一个对象状态发生改变时，其相关依赖对象皆得到通知并被自动更新
  - 观察者模式又叫
    - 发布-订阅（Publish/Subscribe）模式
    - 模型-视图（Model/View）模式
    - 源-监听器（Source/Listener）模式
    - 从属者（Dependents）模式
  - 观察者模式是一种对象行为型模式

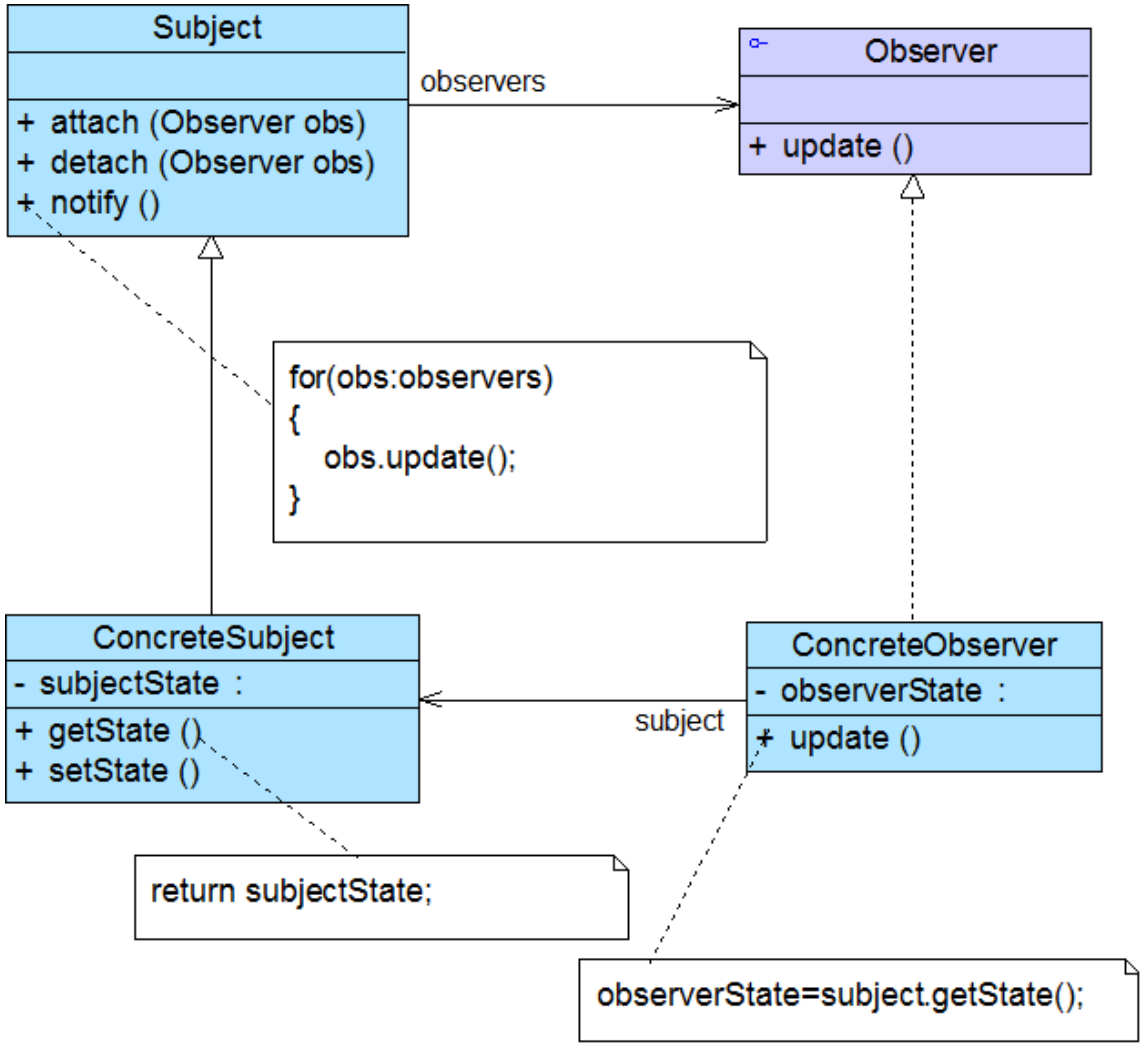


观察者模式

观察者模式包含如下角色：

- Subject: 目标
- ConcreteSubject: 具体目标
- Observer: 观察者
- ConcreteObserver: 具体观察者

模式结构



观察者模式

- 模式分析
  - 观察者模式描述如何建立对象与对象之间的依赖关系，以及如何构造满足这种需求的系统
  - 这一模式中的关键对象是观察目标和观察者，一个目标可以有任意数目的与之相依赖的观察者，一旦目标的状态发生改变，所有的观察者都将得到通知
  - 作为对这个通知的响应，每个观察者都将即时更新自己的状态，以与目标状态同步，这种交互也称为发布-订阅
  - 目标是通知的发布者，它发出通知时并不需要知道谁是它的观察者，可以有任意数目的观察者订阅并接收通知

观察者模式

- 模式分析
  - 客户端代码片段如下所示：

```
Subject subject = new ConcreteSubject();
Observer observer = new ConcreteObserver();
subject.attach(observer);
subject.notify();
```

观察者模式

- 模式分析
  - 典型的抽象目标类代码如下所示：

```
import java.util.*;
public abstract class Subject {
     protected ArrayList observers = new ArrayList();
     public abstract void attach(Observer observer);
     public abstract void detach(Observer observer);
     public abstract void notify();
}
```

观察者模式

- 模式分析
  - 典型的具体目标类代码如下所示：

```
public class ConcreteSubject extends Subject {
    public void attach(Observer observer) {
        observers.add(observer);
    }

    public void detach(Observer observer) {
        observers.remove(observer);
    }

    public void notify() {
        for(Object obs : observers) {
            ((Observer)obs).update();
        }
    }
}
```

观察者模式

- 模式分析
  - 典型的抽象观察者代码如下所示：

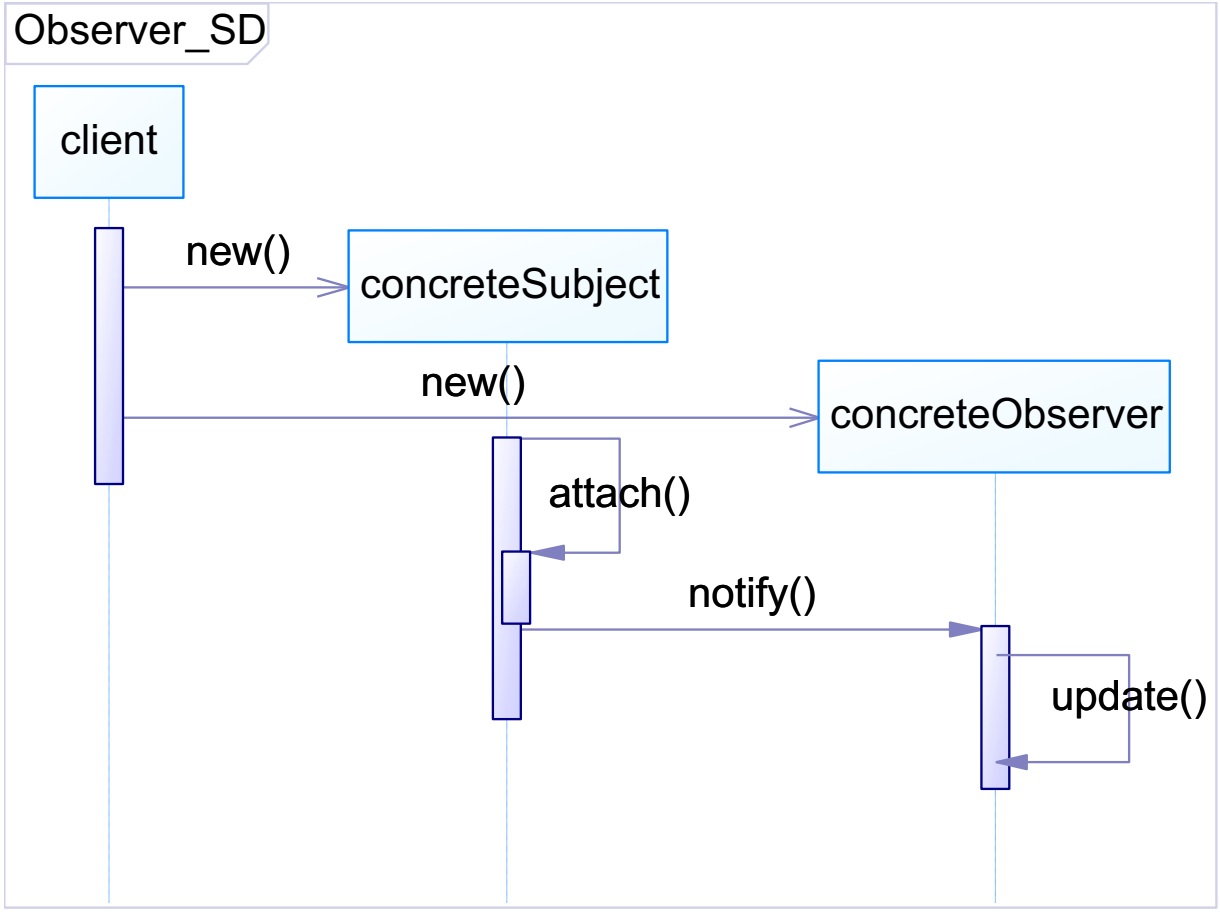
```
public interface Observer {
    public void update();
}
```

  - 典型的具体观察者代码如下所示：

```
public class ConcreteObserver implements Observer {
    public void update() {
        //具体更新代码
    }
}
```

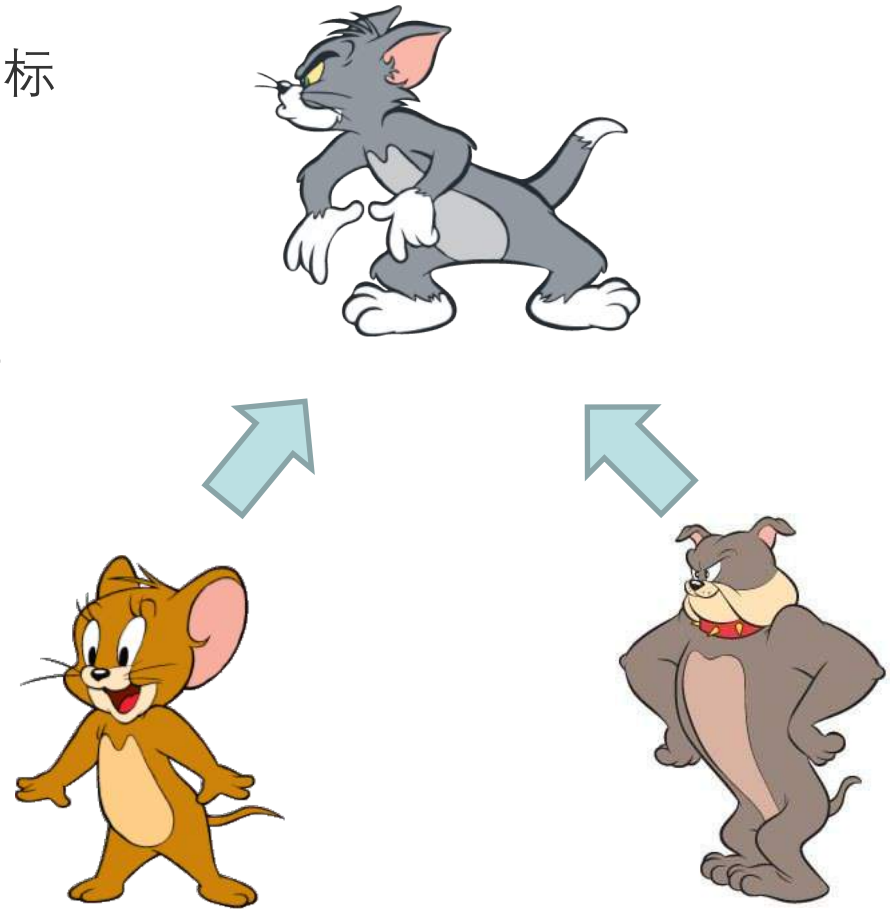
观察者模式

- 模式分析
  - 观察者模式顺序图如下所示：



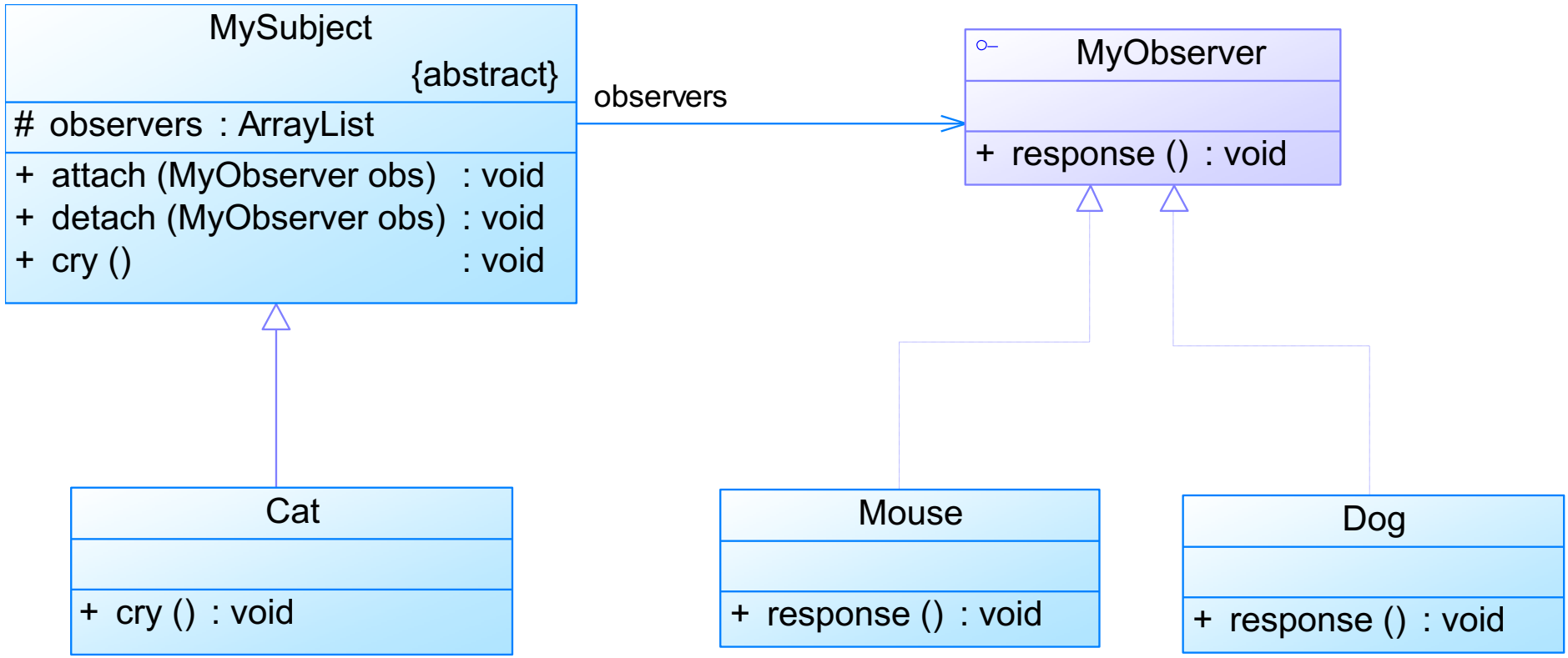
观察者模式

- 观察者模式实例与解析
  - 实例：猫、狗与老鼠
    - 假设猫是老鼠和狗的观察目标
    - 老鼠和狗是观察者
    - 猫叫老鼠跑，狗也跟着叫
    - 使用观察者模式描述该过程



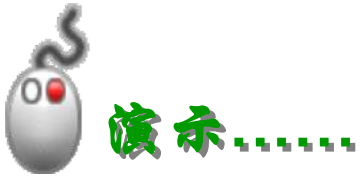
观察者模式

- 观察者模式实例与解析
  - 实例：猫、狗与老鼠



观察者模式

- 观察者模式实例与解析
  - 实例：猫、狗与老鼠
    - 参考代码 （observer）



观察者模式

- 模式优缺点
  - 观察者模式的优点
    - 观察者模式可以实现表示层和数据逻辑层的分离，并定义了稳定的消息更新传递机制，抽象了更新接口，使得可以有各种各样不同的表示层作为具体观察者角色
    - 观察者模式在观察目标和观察者之间建立一个抽象的耦合
    - 观察者模式支持广播通信
    - 观察者模式符合“开闭原则”

观察者模式

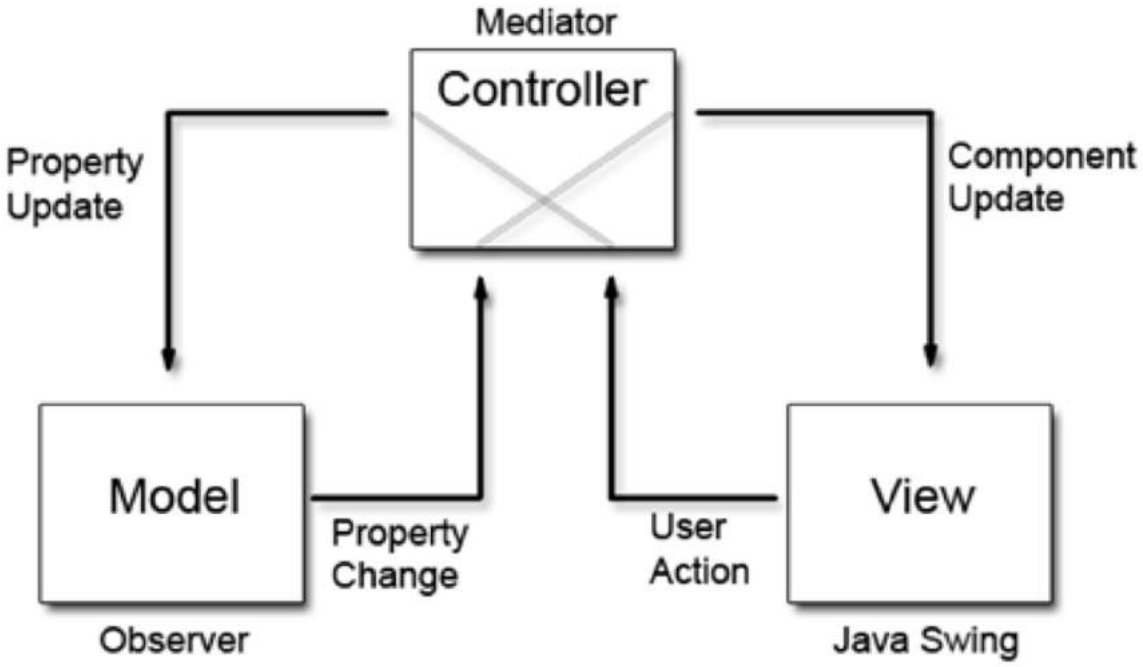
- 模式优缺点
  - 观察者模式的缺点
    - 如果一个观察目标对象有很多直接和间接的观察者的话，将所有的观察者都通知到会花费很多时间
    - 如果在观察者和观察目标之间有循环依赖的话，观察目标会触发它们之间进行循环调用，可能导致系统崩溃
    - 观察者模式没有相应的机制让观察者知道所观察的目标对象是怎么发生变化的，而仅仅只是知道观察目标发生了变化

观察者模式

- 模式扩展
  - MVC模式
    - MVC模式是一种架构模式，包含三个角色：模型（Model），视图（View）和控制器（Controller）。
    - 观察者模式可以用来实现MVC模式，观察者模式中的观察目标就是MVC模式中的模型，而观察者就是MVC中的视图，控制器充当两者之间的中介者。
    - 当模型层的数据发生改变时，视图层将自动改变其显示内容。

观察者模式

- 模式扩展
  - MVC模式



行为型模式

- 行为型模式
  - 职责链模式（Chain of Responsibility）
  - 命令模式（Command）
  - 解释器模式（Interpreter）
  - 迭代器模式（Iterator）
  - 中介者模式（Mediator）
  - 备忘录模式（Memento）
  - 观察者模式（Observer）
  - 状态模式（State）
  - 策略模式（Strategy）
  - 模板方法模式（Template Method）
  - 访问者模式（Visitor）

END





Thanks!